Mindmap — ASCII only (`coding/`)

E/M/H = difficulty · ★ = must-nail · → = one-line hint

coding/data-structures/

advanced.md

```
coding/data-structures/advanced.md
└── advanced.md
    ├── Suffix Array
    │   ├── Number of Distinct Substrings [M]
    │   │   → Build SA via prefix doubling (O(n log² n)) or SA-IS (O(n)). Build LCP
    │   │     via Kasai's O(n) algorithm
    │   └── Longest Common Prefix of Suffixes (LCP Array) [M]
    │       → See implementation in count_distinct_substrings above. Longest
    │         repeated = s[sa[idx] : sa[idx] + max(lcp)]
    ├── LFU Cache — O(1) Implementation
    │   └── LFU Cache [H]
    │       → - get(key): if key missing return -1. Increment freq[key]. Move key
    │         from freq_to_keys[old_freq] to freq_to_keys[new_freq]. Update min_freq
    │         if old_freq == min_freq and that bucket is now empty. - put(key,
    │         value): if key exists, update value and call get logic (increment
    │         freq). If capacity full, evict: call popitem(last=False) (LRU) from
    │         freq_to_keys[min_freq], remove from all maps. Insert new key with
    │         freq=1. Set min_freq=1 (new insertions always land at freq 1)
    ├── Segment Tree
    │   ├── Range Sum Query — Mutable (LC 307) [M]
    │   │   → Tree size 4*n. build fills leaves and merges upward. update walks to
    │   │     the leaf, updates, merges on the way back. query recursively combines:
    │   │     if query fully covers current node, return stored sum; if no overlap,
    │   │     return 0; else recurse into both children
    │   ├── Range Minimum Query (Segment Tree) [M]
    │   │   → Build, update, query are identical to range sum — swap + for min and
    │   │     0 for inf in the no-overlap base case
    │   ├── Count of Smaller Numbers After Self (LC 315) [M]
    │   │   → Coordinate compress first: sort unique values, assign ranks. Use
    │   │     BIT/segment tree of size m. Right-to-left pass: query prefix sum up to
    │   │     rank[i]-1, then increment rank[i]
    │   ├── Number of Longest Increasing Subsequences (LC 673) [M]
    │   │   → For each nums[i] (left to right), query the tree over [0, rank[i]-1]
    │   │     to get the best (len, cnt) for any element smaller than nums[i]. Then
    │   │     new_len = len+1, new_cnt = cnt. Point-update rank[i] with (new_len,
    │   │     new_cnt). Merge rule: keep the entry with larger length; if tie, add
    │   │     counts
    │   └── My Calendar I / II / III (LC 729 / 731 / 732) [M]
    │       → Use a sorted map (balanced BST) as a difference array: +1 at start,
    │         -1 at end. Scan prefix sums to find max overlap. Python uses SortedList
    │         from sortedcontainers or a defaultdict with sorted keys
    ├── Binary Indexed Tree (Fenwick Tree)
    │   ├── Range Sum Query — Mutable (BIT version) [M]
    │   │   → Store original array. On update(i, val): compute delta = val -
    │   │     nums[i], update nums[i], then propagate delta through BIT. sumRange(l,
    │   │     r) = prefix(r+1) - prefix(l)
    │   ├── Count Inversions (Fenwick Tree) [M]
    │   │   → Sort unique values to get ranks. BIT stores counts. After inserting
    │   │     r, query(r) = count of elements ≤ r already inserted. Elements already
    │   │     inserted with rank > r = query(m) - query(r) = inversions contributed
    │   │     by current element
    │   ├── Reverse Pairs (LC 493) [H]
    │   │   → Coordinate-compress all values AND all 2*value together (to handle
    │   │     the 2*nums[j] query correctly). For each nums[i] (right to left): count
    │   │     elements with rank ≤ rank of (nums[i]-1) // 2... Alternatively: use
    │   │     merge sort which is cleaner
    │   └── Number of Subarrays with Bounded Maximum (LC 795) [M]
    │       → Scan once; maintain curr = current run length of elements ≤ k. When
    │         element > k, reset curr to 0. Accumulate curr into total
    ├── Sparse Table (Extended)
    │   ├── Sparse Table for Range Minimum Query (static) [M]
    │   │   → Build with two nested loops. Precompute log2 table to make queries
    │   │     O(1) (no math.log call)
    │   └── Sparse Table RMQ with LCA Application [M]
    │       → DFS to build Euler tour array and first[node] = first occurrence
    │         index. Build sparse table on depths. lca(u, v): query min-depth in
    │         [first[u], first[v]], return that node
    ├── Skip List (Full Implementation)
    │   └── Skip List Insert / Search / Delete [M]
    │       → _find_predecessors(target) traverses from the top level downward,
    │         collecting the rightmost node at each level whose value is < target.
    │         search checks level 0. add generates a random level, inserts node.
    │         erase removes one occurrence
    ├── Monotonic Stack / Queue (Advanced)
    │   ├── Maximum of Subarrays of Size K (Sliding Window Ma... [H]
    │   │   → For each new element: pop from back while nums[back] ≤ nums[i]
    │   │     (they're dominated). Push i. Pop from front if front ≤ i - k (out of
    │   │     window). When i ≥ k-1, nums[dq[0]] is the answer
    │   ├── Sum of Subarray Ranges (LC 2104) [H]
    │   │   → Two monotonic stack passes (one for max boundaries, one for min
    │   │     boundaries). Use strict vs. non-strict inequalities on one side to
    │   │     avoid double-counting duplicates
    │   └── Number of Submatrices That Sum to Target (LC 1074) [M]
    │       → Precompute 2D prefix sums. Outer two loops: fix r1 and r2. Inner
    │         loop: build running column sum, use prefixSum - target in a hash map to
    │         count subarrays
    └── Sqrt Decomposition
        └── Block Decomposition for Range Queries [M]
            → For sumRange(l, r): if l and r are in the same block, iterate
              directly. Otherwise: sum partial left block, sum full middle blocks via
              block_sum, sum partial right block
```

array.md

```
coding/data-structures/array.md
└── array.md
    ├── Two Pointers
    │   ├── Two Sum (sorted variant) [E]
    │   │   → Two Pointers on sorted array: l=0, r=n-1. If numbers[l] + numbers[r]
    │   │     == target, done. If sum < target, l++ (we need larger). If sum >
    │   │     target, r-- (we need smaller). The sorted invariant guarantees we never
    │   │     skip valid pairs
    │   ├── 3Sum [M]
    │   │   → Fix + Two Pointers: Sort. For each i, if nums[i] > 0 break (sorted —
    │   │     no triplet can sum to 0). Skip duplicate i. Run two-pointer on the
    │   │     suffix. On match, skip duplicate left and right before advancing both.
    │   │     Three deduplication sites: i, left, right. Interview note: sorting is
    │   │     what makes the duplicate skipping and early break safe, so this pattern
    │   │     is usually the cleanest solution in interviews
    │   ├── 3Sum Closest [M]
    │   │   → Sort + Two Pointers with closest tracking: Sort. For each i,
    │   │     two-pointer on suffix. Compute s = nums[i]+nums[l]+nums[r]. Update
    │   │     closest if |s - target| < |closest - target|. If s < target, l++. If s
    │   │     > target, r--. If exact match, return immediately
    │   ├── 4Sum [M]
    │   │   → Two nested loops + Two Pointers: Sort. Outer loop i, inner loop j =
    │   │     i+1. Skip duplicate i and j. Two pointers l=j+1, r=n-1. Same pointer
    │   │     logic as 3Sum. Early termination: if the smallest possible sum for the
    │   │     current i is already too large, break; if the largest possible sum is
    │   │     still too small, continue
    │   ├── Container with Most Water [M]
    │   │   → Two Pointers — advance the shorter line: l=0, r=n-1. Compute area.
    │   │     Always advance the pointer pointing to the shorter line — moving the
    │   │     taller line can never increase min(h[l], h[r]) while width also
    │   │     decreases
    │   ├── Trapping Rain Water [H]
    │   │   → Two Pointers — binding constraint side: l=0, r=n-1, l_max=r_max=0. If
    │   │     l_max <= r_max, the left side is the constraint, so water at l is l_max
    │   │     - height[l] and we advance l. Otherwise, the right side is the
    │   │     constraint, so water at r is r_max - height[r] and we decrement r
    │   ├── Remove Duplicates from Sorted Array [M]
    │   │   → Two Pointers — slow/fast write pattern: slow=1. For fast in 1..n-1:
    │   │     if nums[fast] != nums[slow-1], write nums[slow] = nums[fast], slow++.
    │   │     Return slow
    │   └── Next Permutation [M]
    │       → Three-step: find rightmost descent, swap, reverse suffix: 1. Scan
    │         right-to-left to find index i where nums[i] < nums[i+1] (rightmost
    │         ascending pair from the right). 2. If none found, the whole array is
    │         descending — just reverse it. 3. From the right, find the smallest
    │         element greater than nums[i]; swap them. 4. Reverse nums[i+1:] to get
    │         the smallest permutation of the suffix
    ├── Sliding Window
    │   ├── Max Consecutive Ones III [M]
    │   │   → Variable sliding window — zero count tracking: Expand r. If nums[r]
    │   │     == 0, increment zero count. If zeros > k, shrink l until zeros ≤ k
    │   │     (moving l past a zero decrements zero count). Track r - l + 1 as window
    │   │     size
    │   ├── Longest Repeating Character Replacement [M]
    │   │   → Variable window with monotone max_freq trick: Expand r. Update freq
    │   │     map. If (r - l + 1) - max_freq > k, shrink l by one (decrement freq of
    │   │     s[l]). Key insight: max_freq never needs to decrease — a smaller
    │   │     max_freq cannot yield a larger valid window
    │   ├── Subarrays with K Different Integers [M]
    │   │   → Exactly-k = at_most(k) - at_most(k-1): at_most(k) counts subarrays
    │   │     with ≤ k distinct integers. Use standard sliding window: expand right,
    │   │     if distinct count > k shrink left, add r - l + 1 (all subarrays ending
    │   │     at r and starting at l..r). Answer = at_most(k) - at_most(k-1)
    │   ├── Minimum Window Substring [H]
    │   │   → Sliding window with have counter: Build need freq map for t. Expand
    │   │     r. For each char, if its count in window hits required count, have++.
    │   │     When have == len(need), try to shrink: record window, shrink l. If
    │   │     removing s[l] drops a count below required, have--
    │   └── Sliding Window Maximum [H]
    │       → Monotonic decreasing deque: For each r: pop rear of deque while deque
    │         is non-empty and nums[deque[-1]] <= nums[r]. Append r. Pop front if
    │         deque[0] <= r - k (out of window). Once r >= k-1, the front index gives
    │         the current window max
    ├── Prefix Sum
    │   ├── Subarray Sum Equals K [M]
    │   │   → Prefix sum + hash map complement lookup: Scan left to right,
    │   │     maintaining running_sum and a hash map seen of count of each prefix sum
    │   │     seen so far. Seed seen[0] = 1 (empty prefix). For each element, count
    │   │     += seen[running_sum - k]
    │   ├── Subarray Sums Divisible by K [M]
    │   │   → Prefix modulo — same remainder pairs: Scan with running % k. Handle
    │   │     negative modulo: (running % k + k) % k. Seed seen[0] = 1. For each
    │   │     position, count += seen[running % k] before incrementing
    │   ├── Continuous Subarray Sum [M]
    │   │   → Prefix modulo — first-occurrence index map: Seed seen[0] = -1 (empty
    │   │     prefix at index -1). For each index i, compute rem. If rem in seen and
    │   │     i - seen[rem] >= 2, return True. Otherwise, record rem → i only if not
    │   │     already present (want first occurrence)
    │   └── Product of Array Except Self [M]
    │       → Two-pass left/right product accumulation: Pass 1: output[i] = product
    │         of nums[0..i-1]. Pass 2: maintain right = 1, scan right to left,
    │         multiply output[i] *= right, then right *= nums[i]
    ├── Kadane's Algorithm
    │   ├── Maximum Subarray [E]
    │   │   → Kadane's algorithm — local reset on negative prefix: Initialize cur =
    │   │     best = nums[0] (handles all-negative case). For each subsequent
    │   │     element: cur = max(x, cur + x). Update best = max(best, cur)
    │   ├── Maximum Product Subarray [M]
    │   │   → Track max and min simultaneously: At each element x: new max_prod =
    │   │     max(x, max_prod * x, min_prod * x), new min_prod = min(x, max_prod * x,
    │   │     min_prod * x). Use temps to avoid overwriting. Update global best
    │   └── Maximum Circular Subarray Sum (Maximum Sum Circul... [M]
    │       → Kadane's max + Kadane's min on total sum: Edge case: if all elements
    │         are negative, total - kadane_min = 0 (the empty subarray), which is
    │         wrong. In this case, return kadane_max
    ├── Dutch National Flag / Partitioning
    │   ├── Sort Colors [M]
    │   │   → Dutch National Flag — three-pointer partition: lo=0, mid=0, hi=n-1.
    │   │     While mid <= hi: if nums[mid]==0, swap with lo, advance both; if
    │   │     nums[mid]==1, advance mid; if nums[mid]==2, swap with hi, decrement hi
    │   │     — do NOT advance mid (swapped element from hi is unexamined)
    │   └── Find All Numbers Disappeared in an Array [M]
    │       → Sign-flip in-place visited marking: For each value x = abs(nums[i]),
    │         negate nums[x-1]. After marking, collect all indices where value is
```

```
        still positive — those indices +1 are the missing values
├── Boyer–Moore Voting
│   ├── Majority Element [M]
│   │   → Boyer–Moore voting — cancellation argument: This works because:
│   │     imagine each "non-candidate" vote cancels one "candidate" vote. Since
│   │     majority has > n/2 votes, it survives after all cancellations
│   └── Majority Element II [M]
│       → Boyer–Moore with two candidates: Maintain c1, c2, cnt1, cnt2. For
│         each x: if x == c1, cnt1++; elif x == c2, cnt2++; elif cnt1 == 0, c1=x,
│         cnt1=1; elif cnt2 == 0, c2=x, cnt2=1; else cnt1--, cnt2--. Second pass:
│         count actual frequencies and filter > n/3
├── Floyd's Cycle Detection (on Arrays)
│   └── Find the Duplicate Number [M]
│       → Floyd's cycle detection on implicit linked list: Phase 1 — slow =
│         nums[slow], fast = nums[nums[fast]] until they meet (inside cycle).
│         Phase 2 — reset slow = nums[0] (the start), advance both at speed 1;
│         they meet at the cycle entry = duplicate
├── Difference Array
│   ├── Difference Array Pattern [M]
│   │   → Diff array: +v at l, −v at r+1; prefix sum reconstructs range adds
│   ├── Car Pooling [M]
│   │   → Build diff[0..1001]. For each trip: diff[from] += num; diff[to] −=
│   │     num. Reconstruct prefix sum; if any prefix sum > capacity, return False
│   ├── Corporate Flight Bookings [M]
│   │   → Build diff[0..n+1]. For each booking: diff[first] += seats;
│   │     diff[last+1] −= seats. Prefix sum of diff[1..n] is the answer
│   └── Range Addition [M]
│       → Build diff[0..n] (size n+1 to handle r+1 = n). Apply all updates.
│         Prefix-sum diff[0..n−1] in-place
├── Miscellaneous Array Techniques
│   ├── Best Time to Buy and Sell Stock [M]
│   │   → min_price = inf, max_profit = 0. For each price: min_price =
│   │     min(min_price, price), max_profit = max(max_profit, price − min_price)
│   ├── Move Zeroes [M]
│   │   → slow = 0. For each fast: if nums[fast] != 0, set nums[slow] =
│   │     nums[fast], slow++. After the loop, zero out nums[slow..n−1]
│   ├── Two Sum (hash map variant) [E]
│   │   → Hash map complement lookup: For each (i, x): compute complement =
│   │     target − x. If in map, return [map[complement], i]. Else record x → i.
│   │     Checking before recording ensures we don't use same index twice
│   ├── Jump Game II [M]
│   │   → Greedy BFS — level-by-level farthest reach: Advance i from 0 to n−2.
│   │     Update farthest = max(farthest, i + nums[i]). When i == current_end: a
│   │     new jump is needed, jumps++, current_end = farthest. Stop when
│   │     current_end >= n−1
│   ├── Spiral Matrix [M]
│   │   → Boundary shrinking — four-direction cycling: While top <= bottom and
│   │     left <= right: traverse right along top row, top++; traverse down along
│   │     right col, right−−; if top <= bottom, traverse left along bottom row,
│   │     bottom−−; if left <= right, traverse up along left col, left++. The
│   │     inner checks prevent double-counting for single row/col cases
│   ├── Set Matrix Zeroes [M]
│   │   → Use first row/col as markers — O(1) space: Record if row 0 or col 0
│   │     should be zeroed (check for existing zeros). For all other cells, if
│   │     matrix[i][j] == 0, set matrix[i][0] = 0 and matrix[0][j] = 0. Then zero
│   │     out rows and cols using those markers. Finally handle row 0 and col 0
│   │     separately
│   ├── Longest Consecutive Sequence [M]
│   │   → Hash set — start sequences only from minimums: Insert all values into
│   │     a set. For each n, if n−1 not in set (start of sequence), count
│   │     consecutive n, n+1, n+2, ... until gap. Update best length
│   ├── Rotate Array [M]
│   │   → Triple reversal trick: Normalize k = k % n. Reverse nums[0:n].
│   │     Reverse nums[0:k]. Reverse nums[k:n]
│   └── Median of Two Sorted Arrays [H]
│       → Binary search on shorter array partition: Binary search on shorter
│         array (WLOG m <= n). Partition nums1 at i, derive nums2 partition j =
│         half − i. Check: nums1[i−1] <= nums2[j] and nums2[j−1] <= nums1[i].
│         Adjust binary search accordingly. For odd total, median = max(left
│         sides). For even, median = (max(left) + min(right)) / 2
├── Range / Immutable Prefix Queries
│   ├── Range Sum Query — Immutable (LC 303) [M]
│   │   → Prefix sum array — O(1) per query: Precompute prefix[0..n] where
│   │     prefix[0] = 0 and prefix[i] = prefix[i−1] + nums[i−1]. Each query:
│   │     return prefix[right+1] − prefix[left]
│   ├── Range Sum Query 2D — Immutable (LC 304) [M]
│   │   → 2D prefix sum (inclusion-exclusion): sum(r1,c1,r2,c2) =
│   │     prefix[r2+1][c2+1] − prefix[r1][c2+1] − prefix[r2+1][c1] +
│   │     prefix[r1][c1]
│   └── Contiguous Array (LC 525) [M]
│       → Prefix sum with 0→−1 transform + first-occurrence hash map: Seed seen
│         = {0: −1}. For each index i, update running. If running in seen,
│         candidate length = i − seen[running]. Else store seen[running] = i.
│         Never overwrite (want earliest occurrence for max length)
├── Two-pass / Greedy
│   ├── Jump Game (LC 55) [M]
│   │   → Greedy — track farthest reachable index: reach = 0. For each i in
│   │     0..n−1: if i > reach, return False. Update reach = max(reach, i +
│   │     nums[i]). If reach >= n−1 at any point, return True
│   ├── Candy (LC 135) [M]
│   │   → Two-pass greedy — left then right: Initialize all to 1. Left pass
│   │     enforces left-rising constraint. Right pass enforces right-rising
│   │     constraint using max to preserve the larger requirement
│   └── Gas Station (LC 134) [M]
│       → Greedy — reset start on deficit: total = 0, tank = 0, start = 0. For
│         each i: tank += gas[i] − cost[i], total += gas[i] − cost[i]. If tank <
│         0, set start = i + 1, reset tank = 0. Return start if total >= 0 else
│         −1
├── Matrix
│   ├── Rotate Image (LC 48) [M]
│   │   → Transpose then reverse each row: Transpose: for i in range(n): for j
│   │     in range(i+1, n): swap matrix[i][j] and matrix[j][i]. Reverse: for row
│   │     in matrix: row.reverse()
│   └── Search a 2D Matrix (LC 74) [M]
│       → Binary search treating the matrix as a flat sorted array: lo=0,
│         hi=m*n−1. At each mid: val = matrix[mid//n][mid%n]. Compare with
│         target; adjust lo/hi accordingly
├── Intervals
│   ├── Merge Intervals (LC 56) [M]
│   │   → Sort by start, linear merge scan: Sort by start. Initialize merged =
│   │     [intervals[0]]. For each subsequent interval: if interval.start <=
│   │     merged[−1].end, merge by updating merged[−1].end = max(merged[−1].end,
│   │     interval.end). Else append
│   ├── Insert Interval (LC 57) [M]
│   │   → Three-phase linear scan: before, overlap, after: Phase 1: while
│   │     intervals[i].end < new.start, append. Phase 2: while intervals[i].start
```

```
│   │   <= new.end, extend new.start = min(new.start, ...) and new.end =
│   │   max(new.end, ...). Append merged. Phase 3: append remaining
│   └── Non-overlapping Intervals (LC 435) [M]
│       → Greedy — sort by end, keep earliest-ending non-conflicting interval:
│         Sort by end. prev_end = -inf, kept = 0. For each interval: if start >=
│         prev_end, keep it (kept++, update prev_end = end). Else skip (remove
│         it). Answer = n - kept
├── Miscellaneous (Continued)
│   └── First Missing Positive (LC 41) [M]
│       → Index-as-hash — cyclic placement of values in range [1, n]: Swap
│         phase: for each i, while 1 <= nums[i] <= n and nums[nums[i]-1] !=
│         nums[i], swap nums[i] with nums[nums[i]-1]. Scan phase: return first
│         i+1 where nums[i] != i+1, else return n+1
└── Prefix Sum
    ├── Subarray Sum Equals K (with negative numbers) [M]
    │   → Initialize map with {0: 1} (empty prefix). Walk the array
    │     accumulating running_sum; add count_map[running_sum - k] to the answer;
    │     then increment count_map[running_sum]
    ├── Contiguous Array (Equal 0s and 1s) [M]
    │   → Initialize {0: -1}. For each index, compute prefix sum (treating 0 as
    │     -1). If seen before, update max_len = max(max_len, i -
    │     first_seen[prefix]). Otherwise, store first_seen[prefix] = i
    └── Product of Array Except Self (no division) [M]
        → output[i] after left pass = product of nums[0..i-1]. Then walk right
          to left with a right_product variable, multiply output[i] *=
          right_product, then right_product *= nums[i]
```

graph.md

```
coding/data-structures/graph.md
└── graph.md
    ├── BFS on Graphs
    │   ├── Rotting Oranges [M]
    │   │   → Multi-source BFS — simultaneous spread from all rotten sources:
    │   │     Multi-source start avoids O(R × M×N) repeated BFS. Return max_time if
    │   │     fresh == 0, else -1
    │   ├── Number of Islands (BFS) [M]
    │   │   → Seed the queue with the trigger cell; mark visited on enqueue (not
    │   │     dequeue) to prevent duplicate entries. Mutating the grid avoids an
    │   │     extra visited array
    │   ├── Walls and Gates (LC 286) [M]
    │   │   → Only enqueue cells that are INF — this acts as the visited guard. The
    │   │     first time a room is reached is always via its nearest gate
    │   ├── 01 Matrix (LC 542) [M]
    │   │   → Initialize dist matrix with 0 for zeroes, INF for ones. Enqueue all
    │   │     zeroes at start. Only update a cell if current dist > neighbor dist + 1
    │   ├── Word Ladder [H]
    │   │   → BFS on implicit word graph — L×26 mutation enumeration: For each word
    │   │     dequeued, try all single-char mutations; if mutation == endWord,
    │   │     return. Otherwise enqueue if the word is still in the set and then
    │   │     delete it
    │   ├── Shortest Path in Binary Matrix [M]
    │   │   → BFS from (0,0) — 8-directional, mark on enqueue: Mark cells visited
    │   │     by setting to 1 as you enqueue (not after dequeue) to prevent duplicate
    │   │     enqueueing; return distance when (n-1, n-1) is dequeued
    │   ├── Minimum Knight Moves [M]
    │   │   → BFS with symmetry reduction to first quadrant: Use a visited set; use
    │   │     abs values to exploit symmetry; allow coordinates down to -2 (buffer
    │   │     for (0,0)/(1,1) edge cases)
    │   ├── Employee Importance [M]
    │   │   → Hash map + BFS over subordinate ids: Build {id: employee} map; BFS —
    │   │     dequeue id, add importance, enqueue all subordinate ids
    │   ├── Find if Path Exists in a Graph [M]
    │   │   → Union-Find — reachability in O(E α(N)): Path compression + union by
    │   │     rank for optimal performance
    │   └── Find Center of Star Graph [M]
    │       → O(1) — center appears in both the first and second edges: Check if
    │         edges[0][0] is in edges[1]; if so, it's the center; otherwise
    │         edges[0][1] is the center
    ├── DFS on Graphs
    │   ├── Number of Islands [M]
    │   │   → DFS sinking — flood-fill each component, count triggers: Sinking
    │   │     avoids a separate visited array — the mutation is the visit mark.
    │   │     Increment count only on the initial call, not within DFS
    │   ├── Flood Fill [M]
    │   │   → DFS recolor — guard against same-color infinite loop: Guard with if
    │   │     original == color: return image before DFS
    │   ├── Max Area of Island [M]
    │   │   → DFS returning component size — sink inline: max(dfs(r,c) for all r,c)
    │   │     — zero-cost for water cells
    │   ├── Surrounded Regions [M]
    │   │   → Reverse DFS — mark border-safe 'O's, then flip interior: Walk all 4
    │   │     borders, DFS from each 'O' found there
    │   ├── Pacific Atlantic Water Flow [M]
    │   │   → Reverse BFS from both ocean borders — intersect reachable sets:
```

```
    │   │   Reverse BFS condition — expand to neighbors with height >= current
    │   │   height (uphill in the reverse direction)
    │   ├── All Paths From Source to Target [M]
    │   │   → DFS backtracking on DAG — no visited set needed: path.append(nei),
    │   │   recurse, path.pop()
    │   ├── Number of Provinces (LC 547) [M]
    │   │   → Use a visited array instead of mutating the matrix. The matrix is
    │   │   symmetric but you only need to follow one direction per city
    │   ├── Number of Enclaves (LC 1020) [M]
    │   │   → Walk all 4 borders; DFS from each '1' encountered, sinking to 0.
    │   │   After traversal, sum remaining 1-cells
    │   ├── Clone Graph [M]
    │   │   → DFS with original→clone map — register before recursing: Guard if n
    │   │   in visited: return visited[n] handles cycles. Register BEFORE recursing
    │   │   into neighbors
    │   └── Find Eventual Safe States [M]
    │       → Three-color DFS — 0=unvisited, 1=in-progress, 2=safe: Mark gray
    │       before recursing neighbors; mark black after all neighbors are
    │       confirmed safe
    ├── Topological Sort
    │   ├── Course Schedule [M]
    │   │   → Kahn's BFS topo sort — cycle detection via leftover nodes: If
    │   │   completed == numCourses, no cycle
    │   ├── Course Schedule II [M]
    │   │   → Kahn's topo sort — collect removal order: Append node to order as
    │   │   it's dequeued; return order or []
    │   ├── Alien Dictionary [H]
    │   │   → Edge extraction from adjacent word pairs + Kahn's topo sort: Invalid
    │   │   input detection — if word A is a prefix of word B but A appears after B
    │   │   (e.g., "abc" before "ab"), return "" immediately
    │   └── Minimum Number of Vertices to Reach All Nodes [M]
    │       → Nodes with in-degree 0 — the only possible starting set: Collect all
    │       destination nodes from edges — these have in-degree ≥ 1; return all
    │       nodes not in this set
    ├── Shortest Path / Weighted
    │   ├── Network Delay Time (Dijkstra's) [M]
    │   │   → Dijkstra's — min-heap SSSP with stale-entry skip: After Dijkstra,
    │   │   answer = max of all shortest distances; -1 if any node unreached
    │   ├── Swim in Rising Water [M]
    │   │   → Modified Dijkstra — minimize maximum edge weight (bottleneck path):
    │   │   The first time we reach (n-1,n-1), we have the answer
    │   ├── Cheapest Flights Within K Stops [M]
    │   │   → Bellman-Ford with k+1 rounds — copy dist array each round: Copy dist
    │   │   array each round to prevent using edges discovered in the same round
    │   │   (would allow more than 1 edge per round effectively)
    │   ├── Path with Minimum Effort (LC 1631) [M]
    │   │   → First pop of (rows-1, cols-1) from the heap is the answer. Mark
    │   │   visited on pop to avoid reprocessing
    │   └── Shortest Path in a DAG [M]
    │       → Initialize distances, process nodes in topo order, and update dist[v]
    │       = min(dist[v], dist[u] + w) for each edge
    ├── Bipartite / Coloring
    │   ├── Is Graph Bipartite? [M]
    │   │   → BFS 2-coloring — alternating colors, fail on same-color neighbor:
    │   │   Initialize all colors to -1 (uncolored). For each unvisited node, BFS
    │   │   assigning color 0; assign 1 - color[node] to unvisited neighbors;
    │   │   return False if neighbor has same color
    │   └── Possible Bipartition (LC 886) [M]
    │       → People labeled 1..n — initialize color array of size n+1. For each
    │       uncolored node, BFS alternating colors; return False if same-color
    │       conflict found
    ├── Advanced
    │   ├── Redundant Connection [M]
    │   │   → Union-Find — first edge connecting already-connected nodes is
    │   │   redundant: For each edge (a, b): if union(a, b) returns False (same
    │   │   component), return [a, b]
    │   ├── Minimum Spanning Tree (Kruskal's) [M]
    │   │   → Kruskal's — sort edges by weight, greedily add if no cycle: Sort
    │   │   edges by weight; for each edge, union its endpoints if they're in
    │   │   different components; stop after adding V-1 edges
    │   ├── Longest Path in a DAG [M]
    │   │   → Kahn's topo sort + DP — relax dp[v] = max(dp[u] + 1): dp[node] =
    │   │   max(dp[node], dp[prev] + 1) as edges are relaxed during Kahn's
    │   ├── Reconstruct Itinerary (LC 332) [M]
    │   │   → Use a stack-based iterative post-order DFS: push node to result when
    │   │   its adjacency list is exhausted; reverse at the end
    │   ├── Critical Connections / Bridges (LC 1192) [M]
    │   │   → Track parent to avoid treating the tree edge back to parent as a
    │   │   back-edge. Update low[u] = min(low[u], low[v]) after recursing into v;
    │   │   low[u] = min(low[u], disc[v]) for back-edges
    │   └── Minimum Height Trees (LC 310) [M]
    │       → Decrement n by the number of leaves removed each round; stop when n
    │       <= 2 — remaining nodes are the answer
    └── Advanced Graph Algorithms
        ├── Strongly Connected Components — Kosaraju's Algorithm [M]
        │   → Build adjacency list and its transpose. DFS on original, recording
        │   finish order in a stack. Then repeatedly pop from the stack and DFS on
        │   the transposed graph — all reachable unvisited nodes form one SCC
        ├── Minimum Spanning Tree — Prim's Algorithm [M]
        │   → visited set tracks MST nodes. Pop from heap; if already visited,
        │   skip. Otherwise mark visited, add weight to MST cost, push all
        │   unvisited neighbours into the heap
        └── All-Pairs Shortest Path — Floyd-Warshall [M]
            → Initialize dist[i][j] to edge weight if edge exists, 0 if i == j,
            infinity otherwise. Triple loop: for each k, for each i, for each j:
            dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
```

hashing.md

```
coding/data-structures/hashing.md
└── hashing.md
    ├── Complement Map
    │   ├── Two Sum [E]
    │   │   → Single pass. Before storing x, look up target - x. If found, return
    │   │     [seen[complement], i]. Store x → i after checking to avoid using the
    │   │     same index twice
    │   ├── 3Sum (hash-based) [M]
    │   │   → Sort. Skip duplicate values of a. For the inner scan, use a seen set:
    │   │     if target - b in seen, record triplet; else add b to seen
    │   ├── 4Sum (hash-based) [M]
    │   │   → Fix nums[i] and nums[j]. Inner target is target - nums[i] - nums[j].
    │   │     Use seen set for two-sum on remaining elements
    │   └── Max Number of K-Sum Pairs [M]
    │       → Build Counter. For each unique x, pairs formed = min(freq[x], freq[k
    │         - x]) if x != k - x, else freq[x] // 2. Sum all
    ├── Frequency Map
    │   ├── Valid Anagram [E]
    │   │   → Counter(s) == Counter(t). Or use a single 26-element array: increment
    │   │     for s, decrement for t, check all zeros
    │   ├── First Unique Character in a String [M]
    │   │   → Counter in one pass; second pass finds first with count 1. Two O(n)
    │   │     passes
    │   ├── Ransom Note [M]
    │   │   → Use a hash map or Counter; if any needed character drops below zero,
    │   │     return false
    │   ├── Group Anagrams [M]
    │   │   → For each string, compute tuple(sorted(s)) as key, append to
    │   │     defaultdict(list)
    │   ├── Find All Anagrams in a String [M]
    │   │   → Use two Counter maps (window and p). Track have = number of chars
    │   │     where window count equals p count. When have == len(p_count), record
    │   │     the start
    │   ├── Top K Frequent Elements [M]
    │   │   → Counter → buckets list of size n+1 where buckets[f] holds all numbers
    │   │     with frequency f. Iterate from index n down and collect until we have k
    │   │     elements
    │   ├── Subdomain Visit Count [M]
    │   │   → For "9 discuss.leetcode.com" add 9 to discuss.leetcode.com,
    │   │     leetcode.com, and com. Format output as "count domain" strings
    │   ├── Sort Characters by Frequency [M]
    │   │   → Counter, then sort by count descending, rebuild via ch * count
    │   │     concatenation
    │   └── Minimum Window Substring [H]
    │       → Expand right, update have when a char's count first meets
    │         requirement. Shrink left while have == need. Record minimum window
    │         during each valid state
    ├── Prefix Sum + Map
    │   ├── Longest Subarray with Sum K [M]
    │   │   → first_seen = {0: -1}. At index i, if prefix - k in map, update best =
    │   │     max(best, i - first_seen[prefix - k]). Only insert prefix if not
    │   │     already present (preserve earliest index)
    │   ├── Count Number of Nice Subarrays [M]
    │   │   → seen = {0: 1}. Running sum increments by 1 for odd elements, 0 for
    │   │     even. Look up prefix - k in seen before updating map
    │   ├── Binary Subarrays with Sum [M]
    │   │   → Identical to subarray sum equals k. The binary constraint doesn't
    │   │     change the algorithm, only guarantees prefix is non-negative
    │   ├── Subarray Sum Equals K [M]
    │   │   → Maintain running prefix sum. Before updating the map, check
    │   │     seen[prefix - k]. Initialize seen = {0: 1} to handle subarrays starting
    │   │     at index 0
    │   ├── Contiguous Array (Max Equal 0/1 Subarray) [M]
    │   │   → Track running sum with 0→-1 transform. When prefix repeats, the
    │   │     subarray between the two occurrences has sum 0. Store first_seen = {0:
    │   │     -1} and compare i - first_seen[prefix]
    │   ├── Make the Array Sum Divisible by P [M]
    │   │   → rem = sum(nums) % P. If rem == 0, return 0. Use seen = {0: -1}. At
    │   │     each step store prefix % P → i. Look up (prefix - rem) % P
    │   └── Subarray Sums Divisible by K [M]
    │       → seen = {0: 1}. For each element, compute prefix % K (handle
    │         negatives: % K in Python already returns non-negative). Add
    │         seen[(prefix % K)] to count. Increment map
    ├── Design
    │   ├── LRU Cache [M]
    │   │   → On get: look up node, move to front, return val. On put: if key
    │   │     exists update and move to front; else insert at front; if over capacity
    │   │     evict tail
    │   ├── Insert Delete GetRandom O(1) [M]
    │   │   → Insert appends to list and stores index in map. Remove swaps target
    │   │     with last element, updates map for the moved element, pops the list,
    │   │     deletes map entry for removed value
    │   └── Design HashMap [M]
    │       → put scans bucket for existing key (update) or appends. get scans for
    │         key, returns -1 if absent. remove filters out the key
    ├── Set Operations
    │   ├── Contains Duplicate [M]
    │   │   → Single pass: if x in seen return True; else seen.add(x).
    │   │     Short-circuits on first duplicate
    │   ├── Intersection of Two Arrays [M]
    │   │   → set(nums1) & set(nums2) in Python. For an explicit approach: iterate
    │   │     the smaller set, check membership in the larger
    │   └── Two Sum Less Than K [E]
    │       → After sort, l = 0, r = n - 1. Converge inward. Track best = max(best,
    │         sum) when sum < k
    ├── Miscellaneous
    │   ├── Longest Consecutive Sequence [M]
    │   │   → Build set. For each x, if x - 1 not in set, extend the chain x, x+1,
    │   │     x+2, ... while each successor is in the set
    │   ├── Substring with Concatenation of All Words [M]
    │   │   → For each offset in [0, word_len), maintain a sliding window of
    │   │     exactly num_words * word_len characters. Add/remove one word at a time
    │   │     from window ends
    │   └── Max Points on a Line [M]
    │       → For each anchor i, build a slope map. Use gcd to normalize: slope =
    │         (dy // g, dx // g). Handle vertical lines (dx == 0) and same-point
    │         duplicates separately
    ├── Rolling Hash / Dedup
    │   ├── Longest Duplicate Substring (Rabin-Karp) [M]
    │   │   → Hash = sum(ord(s[i]) * base^(L-1-i)) % mod for window. Rolling
    │   │     update: new_hash = (old_hash * base - ord(left) * base^L + ord(right))
    │   │     % mod. Store hashes in a set. Return the window on collision
    │   ├── Find Duplicate File in System [M]
    │   │   → For each entry split on spaces: first token is directory, rest are
```

```
    │   │     name(content) tokens. Extract content between ( and ). Map content →
    │   │     list[full_path]. Return groups with size ≥ 2
    │   └── 4Sum II [M]
    │       → Two nested loops for AB → Counter. Two nested loops for CD → look up
    │         complement. Sum all matching counts
    └── Prefix Sum + Hashing
        ├── Subarray Sums Divisible by K (LC 974) [M]
        │   → Initialize {0: 1}. For each element, compute remainder = running_sum
        │     % k. In Python, % always returns non-negative values, so no adjustment
        │     needed. Add count_map[remainder] to the answer, then increment
        │     count_map[remainder]
        ├── Contiguous Array (LC 525) [M]
        │   → Initialize {0: -1}. For each index i, update prefix. If prefix is in
        │     the map, update max_len = max(max_len, i - first_seen[prefix]).
        │     Otherwise record first_seen[prefix] = i
        └── 4Sum II (LC 454) [M]
            → ab_count = Counter(a + b for a in A for b in B). Then total =
              sum(ab_count[-(c + d)] for c in C for d in D)
```

heap.md

```
coding/data-structures/heap.md
└── heap.md
    ├── Top-K Pattern
    │   ├── Kth Largest Element in a Stream [M]
    │   │   → On each add, push the new value. If heap size exceeds k, pop the
    │   │     minimum. Root is the answer in O(1); each insert is O(log k)
    │   ├── Last Stone Weight [M]
    │   │   → Pop twice, push abs(a - b) if non-zero. Repeat until one or zero
    │   │     stones remain
    │   ├── K Closest Points to Origin [M]
    │   │   → For each point compute x²+y². Push (-dist, x, y) onto a max-heap. If
    │   │     size exceeds k, pop the farthest. Remaining heap contains the k closest
    │   ├── Top K Frequent Elements [M]
    │   │   → Build Counter in O(n). Heap push (freq, num) for each unique num; pop
    │   │     when size > k. Alternatively, bucket sort by frequency for O(n)
    │   ├── Furthest Building You Can Reach [M]
    │   │   → For each upward jump, assign a ladder (push jump to heap). If ladders
    │   │     exhausted, pop the smallest ladder-jump, reclaim it as bricks. If
    │   │     bricks insufficient for the current jump, stop
    │   ├── Kth Largest Element in an Array [M]
    │   │   → For each number, push to heap. If heap exceeds size k, pop the
    │   │     minimum. Root after full pass is the answer in O(1)
    │   └── Top K Frequent Words [M]
    │       → Count with Counter, then ask for the k best items under that custom
    │         key. This is safer than hand-rolling a size-k heap because naive tuple
    │         ordering is easy to get wrong for ties
    ├── Scheduling / Reorganization
    │   ├── Task Scheduler [M]
    │   │   → Count frequencies. Compute max_freq. The formula models "frames" of
    │   │     size n+1 with the most frequent task anchoring each frame. Return
    │   │     max(formula, len(tasks))
    │   └── Reorganize String [M]
    │       → Impossible if max_freq > (len(s) + 1) // 2. Otherwise, greedily fill
    │         from the heap
    ├── Two Heaps
    │   ├── Find Median from Data Stream [M]
    │   │   → Always push to lo, then move lo's max to hi to maintain order.
    │   │     Rebalance sizes so lo is never smaller than hi. Median is either lo[0]
    │   │     or the average of both tops
    │   ├── Sliding Window Median [H]
    │   │   → Maintain lo (max-heap) and hi (min-heap). Slide window: add new
    │   │     element, mark removed element as "invalid". Rebalance heaps. When
    │   │     reading tops, skip invalid elements
    │   └── IPO (Maximize Capital) [M]
    │       → Sort zip(capital, profits) by capital. Use a pointer i advancing when
    │         projects[i][0] <= w. Max-heap holds unlocked profits (negated). Repeat
    │         k times
    ├── K-Way Merge
    │   ├── Merge K Sorted Lists [H]
    │   │   → Seed heap with head of each non-null list. Use list_id as tie-breaker
    │   │     to avoid comparing ListNode objects (not comparable in Python)
    │   ├── Find K Pairs with Smallest Sums [M]
    │   │   → Pop smallest (sum, i, j), record pair. Push (nums1[i] + nums2[j+1],
    │   │     i, j+1) if j+1 < len(nums2). Stop after k pops
    │   ├── Kth Smallest Element in a Sorted Matrix [M]
    │   │   → Push (matrix[i][0], i, 0) for all i. Pop k-1 times advancing
```

```
    │   │     (matrix[i][j+1], i, j+1). The k-th pop is the answer
    │   ├── Smallest Range Covering Elements from K Lists (He... [M]
    │   │   → Seed heap with (lists[i][0], i, 0). Track cur_max = max of all
    │   │     initial first elements. Each pop gives a new candidate min; update
    │   │     range if cur_max - min is smaller. Stop when any list is exhausted
    │   ├── K-th Smallest in M Sorted Arrays [M]
    │   │   → Push (arrays[i][0], i, 0) for all i. Pop and push (arrays[i][j+1], i,
    │   │     j+1) until k pops done
    │   └── Maximum CPU Load [M]
    │       → Sort jobs by start. For each job, pop from heap all jobs with end <=
    │         job.start. Push current job's end time and load. Track running sum of
    │         active loads and record maximum
    ├── Dijkstra / Graph
    │   ├── Network Delay Time [M]
    │   │   → Build adjacency list. Push (0, k). Pop min dist node; skip if already
    │   │     visited. Relax neighbors. Track visited set. Answer =
    │   │     max(dist.values()) if len(dist) == n else -1
    │   ├── Path with Minimum Effort [M]
    │   │   → Push (0, 0, 0). For each pop, update neighbors with max(effort, abs
    │   │     diff). Skip if already visited at a better effort
    │   └── Swim in Rising Water [M]
    │       → Push (grid[0][0], 0, 0). Pop min elevation; if it's the destination
    │         return it. Mark visited. Push unvisited neighbors with max(current_t,
    │         grid[nr][nc])
    ├── Design
    │   ├── Design Twitter [M]
    │   │   → Global time counter increments with each tweet. Each user has a list
    │   │     of (time, tweetId). For feed: seed heap with latest tweet from each
    │   │     followee+self. Pop max; push that user's next tweet. Collect 10
    │   └── Ugly Number II [M]
    │       → Push 1. Pop min (= current ugly). Push val*2, val*3, val*5 if not
    │         seen. Repeat n times
    └── Heap Applications
        ├── Reorganize String (LC 767) [M]
        │   → Alternate approach (cleaner): pop the top character, append it, push
        │     the previous character back (if count > 0). This naturally avoids
        │     placing the same character twice in a row
        ├── IPO — Maximize Capital (LC 502) [M]
        │   → Sort (capital, profit) pairs. Pointer i advances while capital[i] <=
        │     w. After unlocking, pop from the max-heap and add profit to w. Repeat k
        │     times
        └── Minimum Refueling Stops (LC 871) [M]
            → Walk through stations in order. While fuel < station.position -
              current_position and heap is non-empty, pop the largest fuel and add it
              to fuel (increment stops). If still can't reach the next station,
              return -1
```

linked-list.md

```
coding/data-structures/linked-list.md
└── linked-list.md
    ├── In-Place Reversal
    │   ├── Reverse Linked List [E] ★
    │   │   → Three-pointer iterative reversal: Save nxt = curr.next before
    │   │     overwriting. Set curr.next = prev. Advance prev = curr, curr = nxt.
    │   │     When curr is None, prev is the new head. Watch the order carefully:
    │   │     save next before rewiring or you lose the rest of the list
    │   ├── Reverse Linked List II [E]
    │   │   → Find pre-node + in-place splice-reversal: Each iteration: save nxt =
    │   │     curr.next, detach nxt from its position, reattach it after pre. This
    │   │     inserts nodes one by one at the front of the reversed section. After
    │   │     right - left iterations, the segment is reversed. If left == right, the
    │   │     loop runs zero times and the list stays unchanged
    │   ├── Palindrome Linked List [M]
    │   │   → Find middle + reverse second half + compare: Slow/fast to find
    │   │     middle. If the list has odd length, advance slow one more step to skip
    │   │     the middle node. Reverse from that point onward. Walk two pointers —
    │   │     one from head, one from reversed head — checking values. Restore
    │   │     (optional): reverse second half back
    │   ├── Reorder List [M] ★
    │   │   → Find middle + reverse second half + interleave: Slow/fast to find
    │   │     mid. Reverse second half. Merge two halves alternating: take one from
    │   │     first, one from second (reversed), repeat until second half is
    │   │     exhausted
    │   └── Reverse Nodes in K-Group [M]
    │       → Count-verify + in-place reversal per group: Check-count loop +
    │         standard in-place reversal of exactly K nodes. After reversing, head
    │         (original) is the tail of the reversed group; link it to the recursive
    │         result
    ├── Fast / Slow Pointers
    │   ├── Linked List Cycle [M]
    │   │   → Floyd's fast/slow pointer cycle detection: Start both at head. Loop:
    │   │     slow = slow.next, fast = fast.next.next. Check slow is fast (identity,
    │   │     not equality). If fast or fast.next is None, exit — no cycle. The
    │   │     identity check matters whenever node values can repeat
    │   ├── Middle of the Linked List [M]
    │   │   → Fast/slow one-pass middle finder: while fast and fast.next:
    │   │     slow=slow.next, fast=fast.next.next. When loop exits, slow is the
    │   │     middle. For odd-length: exact middle. For even-length: second of the
    │   │     two middles (because fast exhausts before the last step)
    │   ├── Remove Nth Node From End of List [M]
    │   │   → Two pointers with N+1 gap: Dummy → head. Advance fast by n+1 steps.
    │   │     Then while fast: slow=slow.next, fast=fast.next. Now slow is the
    │   │     predecessor of the node to delete. slow.next = slow.next.next
    │   ├── Delete the Middle Node of a Linked List [M]
    │   │   → Slow/fast with prev pointer — land before middle: prev = dummy,
    │   │     advance fast two steps, slow one step, prev follows slow. When fast is
    │   │     None (or fast.next is None), slow is the middle node to delete;
    │   │     prev.next = slow.next
    │   └── Happy Number [M]
    │       → Floyd's cycle detection on the implicit sequence: Define
    │         digit_square_sum. Run slow/fast until slow == fast. If the meeting
    │         value is 1, return True. Else return False
    ├── Floyd's Cycle Detection
    │   ├── Linked List Cycle II [M]
```

```
│  │  → Floyd's two-phase cycle entry detection: Phase 1 — slow and fast meet
│  │    after slow travels distance d+c, fast travels d+c+L (one full cycle
│  │    extra), where d = head-to-entry, c = entry-to-meeting, L = cycle
│  │    length. This implies d = L − c = d' (distance from meeting point back
│  │    to entry). Phase 2 — slow resets to head, both advance 1 step at a time
│  │    → they meet at the entry
│  └── Find the Duplicate Number (Floyd's variant) [M]
│     → Floyd's on implicit linked list defined by array values: Identical to
│       Linked List Cycle II but operating on array indices. slow = nums[slow],
│       fast = nums[nums[fast]]. After meeting, reset slow = nums[0] (not 0,
│       because the linked list starts from nums[0])
├── Merge / Sorting
│  ├── Merge Two Sorted Lists [M]
│  │  → Dummy head + two-pointer merge: dummy → result chain. curr pointer
│  │    builds the result. While both l1 and l2 non-null: attach smaller,
│  │    advance it. After loop, attach the non-null remainder
│  ├── Merge K Sorted Lists [H]
│  │  → Min-heap of size k: Initialize heap with heads of all non-null lists.
│  │    While heap non-empty: pop min node, attach to result, push node.next if
│  │    non-null
│  ├── Sort List [M]
│  │  → Bottom-up merge sort on linked list: For each sublist size size = 1,
│  │    2, 4, 8, ...: split list into pairs of size-length sublists, merge each
│  │    pair, connect results. One full pass per doubling of size, log n passes
│  │    total. This is the version interviewers like when they explicitly ask
│  │    for O(1) extra space
│  └── Insertion Sort List [M]
│     → Dummy head + find-insertion-point per node: Detach each node from the
│       original list. Walk the sorted prefix from dummy until prev.next.val >
│       node.val or prev.next is None. Insert node between prev and prev.next.
│       If the input is nearly sorted, this often behaves closer to linear time
├── Copy / Design
│  ├── Copy List with Random Pointer [M]
│  │  → Hash map original→clone, two-pass wiring: Pass 1: iterate and create
│  │    {node: ListNode(node.val)} for all nodes. Pass 2: for each original
│  │    node, set clone.next = map[node.next], clone.random = map[node.random].
│  │    Mapping None → None keeps the wiring concise
│  └── LRU Cache [M]
│     → Doubly linked list + hash map: Dummy head and dummy tail eliminate
│       all edge cases in _remove and _insert_front. On get: remove from
│       current position, insert at front, return value. On put: if key exists,
│       remove old; create new node, insert at front, update map; if over
│       capacity, remove LRU (tail.prev), delete from map
├── Other Manipulation
│  ├── Add Two Numbers [M]
│  │  → Digit-by-digit addition with carry: carry = 0. While l1 or l2 or
│  │    carry non-zero: sum the current digits (0 if list exhausted) + carry.
│  │    New digit = sum % 10, carry = sum // 10. Append digit node to result
│  ├── Swap Nodes in Pairs [M]
│  │  → Dummy head + iterative pair swapping: Save first = curr.next, second
│  │    = curr.next.next. Then: curr.next = second, first.next = second.next,
│  │    second.next = first. Advance curr = first (first is now behind second
│  │    after swap)
│  ├── Intersection of Two Linked Lists [M]
│  │  → Two pointers traversing both lists — alignment by total distance: p1
│  │    traverses A then B; p2 traverses B then A. When one reaches None,
│  │    redirect to the other list's head. They travel m+n and n+m steps
│  │    respectively — equal total. They meet at the intersection (or both
│  │    reach None simultaneously = no intersection)
│  ├── Rotate List (Circular List Trick) [M]
│  │  → Make circular, find new tail, break circle: Walk to find length and
│  │    tail. Connect tail to head (circular). Walk to position n − k%n − 1 for
│  │    new tail. new_head = new_tail.next. Break: new_tail.next = None
│  ├── Remove Duplicates from Sorted List [M]
│  │  → Single pass: skip consecutive equal nodes: Iterate curr. While
│  │    curr.next exists and curr.next.val == curr.val, set curr.next =
│  │    curr.next.next. After the inner loop, advance curr = curr.next
│  ├── Remove Duplicates from Sorted List II [M]
│  │  → Dummy head + prev pointer skipping duplicate runs: Dummy head. prev =
│  │    dummy. While curr non-null: if curr.next exists and curr.val ==
│  │    curr.next.val, record dup_val, advance curr past all nodes with that
│  │    value, set prev.next = curr.next. Else prev = curr. curr = curr.next
│  ├── Partition List [M]
│  │  → Two dummy heads — collect two sublists, then join: Null-terminate the
│  │    greater chain (greater_tail.next = None) to avoid cycles if the
│  │    original tail landed in the less chain
│  ├── Flatten a Multilevel Doubly Linked List [M]
│  │  → Walk the list. At each node with a child: push node.next to stack,
│  │    set node.next = node.child, fix prev pointers, clear node.child. When
│  │    node.next is None and stack is non-empty, pop and link
│  └── LFU Cache [H] ★
│     → Two hash maps + one frequency-keyed map of doubly linked lists: On
│       get: increment freq, move key from old freq bucket to new freq bucket,
│       update min_freq if old bucket is now empty and min_freq was that old
│       freq. On put: if over capacity, evict from freq_map[min_freq] (popitem
│       from the front = LRU). Then insert at freq=1, set min_freq = 1
└── Design
   └── LRU Cache (Doubly Linked List + Hash Map) [M] ★
      → get: if key missing return -1, else move node to front and return
        value. put: if key exists update value and move to front; if new key
        and at capacity, remove the node just before the tail (LRU), then
        insert new node at front
```

queue.md

```
coding/data-structures/queue.md
└── queue.md
    ├── Monotonic Deque
    │   ├── Sliding Window Maximum [H]
    │   │   → EDGE CASES:: For each index i — (1) evict from the back any index
    │   │     whose value ≤ nums[i] (they are dominated and can never be future
    │   │     maxima; using <= also drops older duplicates); (2) evict from the front
    │   │     if it has fallen outside the window; (3) append i; (4) once i >= k-1,
    │   │     the front of the deque is the answer. Each index is enqueued and
    │   │     dequeued at most once → O(n) total. - EDGE CASES: k = 1 returns the
    │   │     original array; k = len(nums) returns a single maximum
    │   ├── Sliding Window Minimum [M]
    │   │   → EDGE CASES:: Identical to the maximum variant; flip one comparison —
    │   │     evict from the back when nums[back] >= x instead of <=. The front
    │   │     always holds the index of the current window minimum. Using >= keeps
    │   │     the deque short by discarding older duplicates. - EDGE CASES: k = 1
    │   │     returns the original array; duplicate values are safe because the deque
    │   │     stores indices, not values
    │   ├── Shortest Subarray with Sum at Least K [M]
    │   │   → EDGE CASES:: Compute prefix sums. Maintain a deque of indices with
    │   │     strictly increasing prefix-sum values. For each j: while the front of
    │   │     the deque satisfies prefix[j] - prefix[front] >= k, update the answer
    │   │     with j - front and pop the front (we want the smallest valid j - i, so
    │   │     once a shorter subarray is found the front is no longer useful). Then
    │   │     maintain the increasing invariant by popping from the back while
    │   │     prefix[back] >= prefix[j], and append j. - EDGE CASES: Keep prefix[0] =
    │   │     0 so subarrays starting at index 0 are handled naturally; if no
    │   │     qualifying subarray exists, return -1
    │   ├── Jump Game VI (DP + Sliding Window Max) [M]
    │   │   → dp[i] = nums[i] + max(dp[j] for j in range(max(0, i-k), i)). Use a
    │   │     deque of indices in decreasing dp value order. Before computing dp[i],
    │   │     evict indices outside the window [i-k, i-1] from the front. The front
    │   │     of the deque is argmax dp in the window
    │   └── Maximum of Minimums of Every Window Size [M]
    │       → 1. Use monotonic stack (increasing) to compute left[] (previous
    │         smaller element index, defaulting to -1) and right[] (next smaller
    │         element index, defaulting to n). 2. For each i, window_size = right[i]
    │         - left[i] - 1; update ans[window_size] = max(ans[window_size],
    │         nums[i]). 3. Suffix maximum pass: for k from n-1 down to 1: ans[k] =
    │         max(ans[k], ans[k+1]). This handles the fact that the minimum of a
    │         larger window is ≤ the minimum of a smaller window — so a value that is
    │         the minimum for window size w is also a candidate for all smaller sizes
    ├── BFS / Level-order
    │   ├── Binary Tree Level Order Traversal [M]
    │   │   → Enqueue root. At the start of each BFS iteration snapshot size =
    │   │     len(queue). Dequeue exactly size nodes, collect their values, enqueue
    │   │     their children. Append the level list to results
    │   ├── Binary Tree Zigzag Level Order Traversal [M]
    │   │   → Toggle left_to_right after each level. When False, reverse the
    │   │     collected level list. Children are always enqueued left-to-right; only
    │   │     the output list is reversed
    │   └── Binary Tree Right Side View [M]
    │       → Standard level-size snapshotting. After processing all nodes in a
    │         level, the most recently processed node value is the rightmost — append
    │         it to results
    ├── BFS Multi-Source
    │   ├── Rotting Oranges [M]
    │   │   → Count fresh oranges. Run BFS — each time a fresh orange is infected,
    │   │     decrement fresh count and enqueue the new cell with time + 1. After
    │   │     BFS, if fresh > 0, return -1 (blocked cells remain). Otherwise return
    │   │     the last timestamp used
    │   ├── 01 Matrix [M]
    │   │   → Initialize all 0 cells with distance 0 in the queue. Initialize all 1
    │   │     cells with inf. BFS outward — first time a 1 cell is reached sets its
    │   │     distance. BFS guarantees minimum distance
    │   ├── Walls and Gates [M]
    │   │   → Enqueue all cells with value 0 (gates). BFS outward — each INF cell
    │   │     reached gets distance = parent's distance + 1. Walls (-1) are never
    │   │     enqueued or updated
    │   ├── Farthest Building from Land (As Far from Land as ... [M]
    │   │   → Seed the queue with all 1 cells at distance 0. BFS outward filling 0
    │   │     cells. Track the last distance assigned — that is the answer
    │   ├── Shortest Path in Binary Matrix [M]
    │   │   → If start or end is 1, return -1 immediately. BFS with 8 directions.
    │   │     The first time (n-1, n-1) is dequeued, return the current distance + 1
    │   └── Minimum Knight Moves [M]
    │       → Reflect (x, y) to (|x|, |y|) — knight distances are symmetric. BFS
    │         from (0,0) inside a bounded box [-2..x+2] × [-2..y+2]; the +2 buffer
    │         handles the small detours needed near the origin
    ├── BFS Single-Source
    │   ├── Word Ladder [H]
    │   │   → Enqueue (beginWord, 1). For each dequeued word, generate all
    │   │     one-letter variants; if a variant is in the word set, enqueue it with
    │   │     distance + 1 and remove from the set. Return the distance when endWord
    │   │     is reached
    │   ├── Open the Lock [M]
    │   │   → Each state has 8 neighbors (4 wheels × 2 directions). Mark deadends
    │   │     and the start as visited before BFS begins. The first time target is
    │   │     reached, return the current depth
    │   ├── Bus Routes [M]
    │   │   → Build stop → [bus_indices] map. BFS starts from all buses that
    │   │     include source. For each bus dequeued, visit all its stops; if target
    │   │     is reached, return bus count. For each stop on this bus, enqueue all
    │   │     other buses that serve it and haven't been visited. Mark buses as
    │   │     visited to avoid re-boarding
    │   ├── Shortest Path in Grid with Obstacles Elimination [M]
    │   │   → Enqueue (0, 0, k) with 0 steps. For each cell, try all 4 neighbors —
    │   │     if a neighbor is free, step to it; if it's an obstacle and remaining_k
    │   │     > 0, step to it and decrement k. Mark (r, c, k) as visited — not just
    │   │     (r, c)
    │   ├── Cheapest Flights Within K Stops [M]
    │   │   → Use Bellman-Ford with exactly k+1 relaxation rounds. Maintain
    │   │     prices[node] = cheapest price to reach node using at most current_round
    │   │     hops. Use a copy of prices per round to avoid using within-round
    │   │     updates
    │   └── Jump Game III [M]
    │       → Enqueue start. For each index, compute both jump targets. If in
    │         bounds and unvisited, enqueue. Return True immediately if a target
    │         index has value 0
    ├── Topological Sort (Kahn's BFS)
    │   ├── Course Schedule [M]
    │   │   → Build adjacency list and in-degree array. Enqueue all nodes with
    │   │     in-degree 0. For each dequeued node, decrement neighbors' in-degrees;
    │   │     enqueue any that reach 0. Count processed nodes — if count equals
```

```
    │   │   numCourses, return True
    │   ├── Course Schedule II [M]
    │   │   → Append each dequeued node to order. If len(order) == numCourses, the
    │   │     graph is a DAG and order is a valid schedule. Otherwise return []
    │   └── Alien Dictionary [H]
    │       → For each adjacent word pair, find the first differing character —
    │         that gives an edge. If word A is a prefix of word B but appears after
    │         B, return "" (invalid). Run Kahn's BFS. If all characters are
    │         processed, the BFS output is the alien alphabet order
    ├── Design
    │   ├── Design Hit Counter [M]
    │   │   → hit: append timestamp. getHits: evict front while front <= timestamp
    │   │     - 300, then return len(deque). This assumes calls arrive with
    │   │     non-decreasing timestamps, which is the usual interview contract
    │   ├── Design Circular Queue [M]
    │   │   → Enqueue writes to (front + size) % cap and increments size. Dequeue
    │   │     advances front by 1 (mod cap) and decrements size
    │   ├── Moving Average from Data Stream [M]
    │   │   → Append each new value to the deque and add to sum. If the deque
    │   │     exceeds size, pop from the front and subtract from sum. Return sum /
    │   │     len(deque)
    │   ├── Number of Recent Calls [M]
    │   │   → Append t. Pop from front while front < t - 3000. Return len(deque)
    │   ├── Implement Queue Using Stacks [M]
    │   │   → Each element moves from push → pop exactly once, so amortized O(1)
    │   │     per dequeue
    │   └── Implement Stack Using Queues [M]
    │       → Push is O(n); pop and top are O(1). (Trade-off: opposite of
    │         queue-from-stacks.)
    ├── Priority Queue / Heap
    │   ├── Task Scheduler [M]
    │   │   → Build frequency map; push all (-count,) entries to the heap. At each
    │   │     time tick: if the cooldown queue front is ready (available_at <= time),
    │   │     push it back onto the heap. If the heap is non-empty, pop and execute
    │   │     the most frequent task, push it to the cooldown queue with updated
    │   │     count. Otherwise, idle. Increment time
    │   └── Find Median from Data Stream [M]
    │       → addNum: push to lo (negate for max-heap), then balance by moving lo's
    │         max to hi if lo[0] > hi[0] or sizes diverge. Rebalance so lo is never
    │         smaller than hi
    ├── Sliding Window with Queue
    │   └── Sliding Window Median (LC 480) [M]
    │       → Lazy deletion: track counts of "dead" elements in each heap. When
    │         computing the median, first pop any dead elements from the heap tops.
    │         Balance rule: len(lo) == len(hi) or len(lo) == len(hi) + 1
    └── Scheduling
        └── Task Scheduler with Cooldown (LC 621) [M]
            → Count frequencies with a Counter. max_freq = highest frequency.
              max_count = number of tasks that share that frequency. Formula accounts
              for max_freq - 1 complete cycles, each of length n+1, plus the final
              partial cycle of max_count tasks
```

segment-tree.md

```
coding/data-structures/segment-tree.md
└── segment-tree.md
    ├── Common Interview Patterns
    │   ├── Range Minimum Query with Lazy Propagation [M]
    │   │   → EDGE CASES:: - Range update [ul, ur] with delta: if node's interval
    │   │     is fully covered, add delta to node.min_val and node.lazy; otherwise
    │   │     push_down first, recurse on children, pull up. - Push down: apply
    │   │     parent.lazy to both children (add to their min_val and lazy), then
    │   │     clear parent's lazy. - Range min query: standard decomposition, same as
    │   │     sum query but return min of left/right. - EDGE CASES: Never push past a
    │   │     leaf; handle the empty-array case up front
    │   └── Range Maximum Query [M]
    │       → EDGE CASES:: Identity for max is -inf (out-of-range nodes return
    │         this). Merge: max(left_child, right_child). - EDGE CASES: Empty arrays
    │         should return -inf for queries or be guarded explicitly, depending on
    │         the API
    ├── Segment Tree Applications
    │   ├── Count of Range Sum [M]
    │   │   → ALTERNATIVE:: Merge sort approach — during merge, for each right-half
    │   │     element prefix[j], count how many left-half elements fall in [prefix[j]
    │   │     - upper, prefix[j] - lower] using two pointers. O(n log n). -
    │   │     ALTERNATIVE: Coordinate compression + BIT works too, especially if you
    │   │     already have a Fenwick template in an interview
    │   ├── Number of Longest Increasing Subsequence [M]
    │   │   → O(n²) DP is simple. Segment tree on values (coordinate compressed)
    │   │     can reduce to O(n log n): tree node stores (max_length, total_count)
    │   │     for values processed so far; query [0, nums[i]-1] for best, then update
    │   │     at nums[i]
    │   └── Queue Reconstruction by Height [M]
    │       → This is a greedy O(n²) insertion. BIT/segment tree can optimize to
    │         O(n log n) by tracking the k-th empty slot
    ├── Binary Indexed Tree (Fenwick Tree)
    │   ├── Count of Smaller Numbers After Self [M]
    │   │   → Coordinate compression maps values to 1..n. BIT query at rank-1 =
    │   │     count of right-side elements smaller than current. Then insert current
    │   │     into BIT
    │   └── Reverse Pairs [M]
    │       → Coordinate compress nums (and also nums[i]//2 values for query). Two
    │         separate coordinate sets or unified set with both values
    ├── Segment Tree — Interval / Scheduling Problems
    │   ├── My Calendar I (LC 729) [M]
    │   │   → Two intervals [s1, e1) and [s2, e2) overlap iff s1 < e2 and s2 < e1.
    │   │     For a new booking [s, e): - Find the insertion point i via bisect_left
    │   │     on starts. - Check left neighbor (i-1): does it end after s? - Check
    │   │     right neighbor (i): does it start before e? - If neither overlaps,
    │   │     insert
    │   ├── My Calendar II (LC 731) [M]
    │   │   → For new [s, e): 1. Check if [s, e) overlaps any interval in overlaps.
    │   │     If yes → return False. 2. Else: add intersection of [s, e) with each
    │   │     existing booking in calendar to overlaps. 3. Add [s, e) to calendar
    │   ├── My Calendar III (LC 732) [M]
    │   │   → Two approaches — difference array / sorted map sweep (simpler, O(n)
    │   │     per query) or segment tree with lazy propagation (O(log MAX) per
    │   │     query). - HOW (difference array with SortedDict): - On book(s, e):
    │   │     diff[s] += 1, diff[e] -= 1. Sweep prefix sum, track max. O(n) per call.
    │   │     - HOW (segment tree): - Range add [start, end-1] += 1. Query global max
```

```
    │   │   (root value). O(log MAX) per call
    │   ├── The Skyline Problem (LC 218) [M]
    │   │   → - Create events: (left, -height, right) for building starts (negative
    │   │     height for sort order), (right, 0, 0) for building ends. - Sort all
    │   │     events by x, then by height (starts before ends at same x — negative
    │   │     heights sort first). - Use a max-heap of (-height, right) for active
    │   │     buildings. - At each x: remove expired buildings (right ≤ current x)
    │   │     from heap top. Current height = -heap[0][0]. If height changed from
    │   │     previous, add [x, height] to result
    │   └── Falling Squares (LC 699) [M]
    │       → For each square: query max height in its interval, compute new height
    │         = query result + size, update the interval to that new height, record
    │         the global max
    └── Segment Tree Applications
        ├── Count of Smaller Numbers After Self (LC 315) — BI... [M]
        │   → Sort unique values to build rank map. Scan right to left: count[i] =
        │     bit.query(rank[nums[i]] - 1). Then bit.update(rank[nums[i]], 1)
        └── Range Sum Query 2D — Mutable (LC 308) [M]
            → Update (r, c) by delta: iterate i = r+1 by i += i & -i, and for each
              i iterate j = c+1 by j += j & -j, adding delta to bit[i][j]. Query
              prefix sum up to (r, c): sum over all i and j indices descending by
              clearing the lowest bit
```

stack.md

```
coding/data-structures/stack.md
└── stack.md
    ├── Monotonic Stack — Next Greater/Smaller
    │   ├── Daily Temperatures [M]
    │   │   → Push index i onto the stack. When temps[i] > temps[stack[-1]], pop
    │   │     and record result[popped] = i - popped. Stack holds indices of
    │   │     temperatures that haven't yet seen a warmer day
    │   ├── Next Greater Element II [M]
    │   │   → Initialize result = [-1] * n. Iterate 2n times. Use i % n to access
    │   │     elements. Only push i % n when i < n
    │   ├── Online Stock Span [M]
    │   │   → Pop all (p, s) where p <= current_price, accumulating their spans.
    │   │     Push (current_price, accumulated_span + 1)
    │   └── Sum of Subarray Minimums [M]
    │       → Use monotonic increasing stack twice (or once with careful boundary
    │         tracking). Left boundary: strict < comparison; right boundary: <= to
    │         avoid double-counting equal elements
    ├── Monotonic Stack — Histogram / Rectangle
    │   ├── Largest Rectangle in Histogram [M]
    │   │   → Append sentinel 0 to flush the stack. On pop, height =
    │   │     heights[popped]. Width = i - stack[-1] - 1 if stack is non-empty, else
    │   │     i (the bar is the global minimum so far)
    │   ├── Maximal Rectangle [M]
    │   │   → heights[j] = heights[j] + 1 if matrix[row][j] == '1' else 0. Run
    │   │     largest_rectangle_area(heights) for each row
    │   └── Trapping Rain Water (stack approach) [H]
    │       → Push indices onto a decreasing stack. On pop (taller bar arrived),
    │         compute bounded water above the popped bar
    ├── Valid Parentheses / Nesting
    │   ├── Valid Parentheses [M]
    │   │   → Map ')' → '(', etc. If stack is empty when closing bracket arrives,
    │   │     or top doesn't match, return False. Valid iff stack is empty at end
    │   ├── Decode String [M]
    │   │   → Parse digits to get k. On [: push (current_string, k), reset both. On
    │   │     ]: pop (prev_str, k), set current = prev_str + current * k. Characters
    │   │     append to current
    │   ├── Remove All Adjacent Duplicates in String [M]
    │   │   → Linear scan. Maintain stack. At each character, cancel with top if
    │   │     equal. Analogous to bracket matching
    │   └── Remove K Digits [M]
    │       → Build the stack left to right. Pop larger elements while k > 0. If k
    │         still > 0 after the loop, trim last k digits from the stack (which is
    │         already sorted ascending). Strip leading zeros
    ├── Stack Design
    │   ├── Min Stack [M]
    │   │   → On push, min_stack pushes min(val, min_stack[-1]). On pop, both
    │   │     stacks pop. get_min returns min_stack[-1]
    │   └── Maximum Frequency Stack [M]
    │       → Push: increment freq[val], append val to group[freq[val]], update
    │         max_freq. Pop: take from group[max_freq], decrement freq[val],
    │         decrement max_freq if the bucket is now empty
    ├── Queue from Stacks
    │   ├── Implement Queue using Stacks [M]
    │   │   → push appends to in_stack. pop/peek: if out_stack is empty, move all
    │   │     of in_stack to out_stack (reversal). Then operate on out_stack.
    │   │     Amortized O(1)
    │   │
```

```
    └── Validate Stack Sequences [M]
        → Maintain pointer j into popped. After pushing pushed[i], pop while
          stack and stack[-1] == popped[j], incrementing j. Valid iff stack is
          empty at end
├── Expression Evaluation
    ├── Evaluate Reverse Polish Notation [M]
    │   → Pop b then a (order matters for - and /). Apply operator. Push
    │     result. Division truncates toward zero: int(a / b) not a // b (handles
    │     negatives)
    ├── Basic Calculator I [M]
    │   → Parse multi-digit numbers. Accumulate into result using current sign.
    │     Handle ( and ) for scope management
    ├── Basic Calculator II [M]
    │   → Parse number, apply prev_op with stack. Default prev_op = '+'. Final
    │     answer = sum of stack
    └── Basic Calculator III [M]
        → Implement a helper that processes until end or ). Inside, use the
          stack-based approach from Calculator II. On (, recurse for the inner
          expression
├── Simulation / Other
    ├── Exclusive Time of Functions [M]
    │   → Parse each log. On "start": if stack non-empty, add elapsed time to
    │     top's exclusive time. Push current. On "end": pop, compute time, add to
    │     result. Update prev_time = end + 1
    ├── Simplify Path [M]
    │   → Split on /. For each part: skip empty strings and .; pop on .. (if
    │     stack non-empty); push otherwise. Join with / and prepend /
    ├── Baseball Game [M]
    │   → Parse each op. Integer: push. +: push stack[-1] + stack[-2]. D: push
    │     stack[-1] * 2. C: pop. Sum the stack at the end
    └── Asteroid Collision [M]
        → While collision conditions hold: if top is smaller, pop (top
          destroyed, current continues); if equal, pop and break (both
          destroyed); if top is larger, break (current destroyed, don't append).
          Use while...else to append only if current survived
├── Parentheses — Score and Repair
    ├── Score of Parentheses (LC 856) [M]
    │   → Start with [0]. On (: append 0. On ): v = stack.pop(); stack[-1] +=
    │     max(2*v, 1). Return stack[0]
    ├── Minimum Add to Make Parentheses Valid (LC 921) [M]
    │   → Final answer = open + close
    └── Check if Word is Valid After Substitutions (LC 1003) [M]
        → After each push check if stack[-3:] == ['a','b','c'] and pop three.
          Valid iff stack is empty at end
├── Monotonic Stack — Arrays and Sequences
    ├── 132 Pattern (LC 456) [M]
    │   → third = -inf. For each element right to left: if num < third, return
    │     True. While stack and stack[-1] < num, set third = stack.pop(). Push
    │     num
    ├── Car Fleet (LC 853) [M]
    │   → Iterate sorted arrival times. Push if > stack[-1] (or stack empty).
    │     Stack size = number of fleets
    ├── Maximum Width Ramp (LC 962) [M]
    │   → Build decreasing stack in one pass. Reverse scan: greedily pop all
    │     valid left endpoints
    ├── Number of Visible People in a Queue (LC 1944) [M]
    │   → Process right to left. For each person, pop from the decreasing stack
    │     while top < current height, incrementing count. Add 1 if stack
    │     non-empty (blocked by a taller person). Push current height
    └── Buildings With an Ocean View (LC 1762) [M]
        → Walk right to left. If heights[i] > max_right, append i to results
          and update max_right. Reverse the results before returning (indices
          must be in ascending order)
├── Iterative Tree Traversal
    ├── Flatten Binary Tree to Linked List (LC 114 — iter... [M]
    │   → Push root. While stack: pop node, if node.right exists push it, if
    │     node.left exists push it. Set node.right = stack[-1] if stack else
    │     None, node.left = None
    └── Path Sum II (LC 113 — iterative DFS) [M]
        → Push (root, target, []). On each pop: if leaf and remaining == 0, add
          copy of path to results. Push right child, then left child (left
          processed first) with updated remaining and path
└── Monotonic Stack — Advanced
    └── Sum of Subarray Minimums (LC 907) [H]
        → In one pass, use the stack to track unresolved indices. When nums[i]
          is smaller than the stack top, pop and compute the contribution of the
          popped element with i as its right boundary. Left boundary comes from
          the new stack top (or -1 if empty)
```

string.md

```
coding/data-structures/string.md
└── string.md
    ├── Frequency Map / Anagram
    │   ├── Valid Anagram [E]
    │   │   → Build Counter(s) and Counter(t). Return Counter(s) == Counter(t). For
    │   │     pure lowercase ASCII, use [0]*26 and compare arrays — same asymptotic
    │   │     cost, better constant
    │   ├── Group Anagrams [M]
    │   │   → For each word, compute key = tuple(freq_array) or key =
    │   │     "".join(sorted(word)). Append word to groups[key]. Return
    │   │     list(groups.values())
    │   └── Find All Anagrams in a String [M]
    │       → Build p_freq. Maintain w_freq for the window. Add the incoming
    │         character on the right; evict the outgoing character on the left when
    │         the window exceeds len(p). If arrays match, record the left index. Use
    │         a matches counter to avoid O(26) comparison: track how many of the 26
    │         buckets currently match between w_freq and p_freq
    ├── Two Pointers — Palindrome
    │   ├── Valid Palindrome [M]
    │   │   → While l < r: skip l while not alphanumeric, skip r while not
    │   │     alphanumeric. If s[l].lower() != s[r].lower(), return False. Advance
    │   │     both pointers inward
    │   ├── Longest Palindromic Substring [M]
    │   │   → expand(l, r) expands while s[l] == s[r] and indices are in bounds,
    │   │     returning the palindrome substring. For each index i, try both
    │   │     expand(i, i) (odd length) and expand(i, i+1) (even length). Track the
    │   │     longest result
    │   └── Palindromic Substrings (Count All Palindromic Sub... [M]
    │       → For each of the 2n - 1 centers, expand while s[l] == s[r]. Each valid
    │         (l, r) pair is one palindromic substring — increment count by 1 each
    │         step
    ├── Sliding Window
    │   ├── Longest Substring Without Repeating Characters [M]
    │   │   → For each right: if s[right] is in the set, remove s[left] and advance
    │   │     left until the duplicate is gone. Then add s[right] and update max
    │   ├── Minimum Window Substring [H]
    │   │   → Expand right: add s[right] to have; if have[c] == need[c], increment
    │   │     formed. When formed == len(need) (window valid): record min window,
    │   │     shrink from left — decrement have[s[left]]; if it drops below
    │   │     need[s[left]], decrement formed. Repeat shrinking until no longer valid
    │   └── Longest Repeating Character Replacement [M]
    │       → Expand right: update count[s[right]] and max_freq. If (window_size -
    │         max_freq) > k, slide left by 1 — don't shrink, just slide. The window
    │         grows when a valid longer window is found
    ├── Prefix Sum on Characters
    │   ├── Prefix Sum on Characters Pattern [M]
    │   │   → Prefix count per char; window validity = freq map compare
    │   ├── Number of Substrings Containing All Three Characters [M]
    │   │   → Maintain last = [-1, -1, -1] for a/b/c. For each r, update
    │   │     last[ord(s[r]) - ord('a')] = r. Add min(last) + 1 to answer (clamped to
    │   │     0 when any char unseen)
    │   ├── Count Vowel Substrings of a Word [M]
    │   │   → atMost(k): l = 0, freq = {}. For each r: if s[r] not a vowel, reset
    │   │     window (l = r+1, freq = {}). Else update freq[s[r]]. While len(freq) >
    │   │     k, shrink l. Add r - l + 1. Answer = atMost(5) - atMost(4)
    │   ├── Binary Subarrays With Sum [M]
    │   │   → seen = {0: 1}, running = count = 0. For each x: running += x, count
    │   │     += seen.get(running - goal, 0), seen[running] = seen.get(running, 0) +
    │   │     1
    │   └── Subarray Sum Equals K (character version) [M]
    │       → For each character ch: running += (1 if ch == target else 0). count
    │         += prefix_count[running - k]. prefix_count[running] += 1
    ├── Encoding / Hashing
    │   ├── Encode and Decode Strings [M]
    │   │   → Encode: for each string, emit str(len(s)) + '#' + s. Decode: read
    │   │     digits up to # to get length L; read exactly L characters as the next
    │   │     string; advance pointer past them; repeat
    │   └── String Hashing (Duplicate Substring Detection) [M]
    │       → Compute hash(s[0..L-1]). For each subsequent position, update the
    │         hash by the sliding formula. Store hashes in a set. On collision,
    │         verify with string comparison to rule out false positives
    ├── Parsing / Simulation
    │   ├── String to Integer (atoi) [M]
    │   │   → Advance i past spaces. Read sign. Accumulate result = result * 10 +
    │   │     digit for each digit character. Before adding, check for overflow: if
    │   │     result > (INT_MAX - digit) // 10, clamp and return. Return sign *
    │   │     result
    │   └── Longest Common Prefix [M]
    │       → Take strs[0] as the reference. For each character index i in strs[0],
    │         check every other string at position i. If any string is shorter than i
    │         or differs at i, return strs[0][:i]
    ├── Two Pointers — Reverse / Subsequence
    │   ├── Reverse String [M]
    │   │   → while l < r: s[l], s[r] = s[r], s[l]; l += 1; r -= 1
    │   └── Is Subsequence [M]
    │       → Single pass through t. If i reaches len(s), all characters matched —
    │         return True
    ├── Parsing / Simulation (Extended)
    │   ├── Decode String [M]
    │   │   → Two stacks (count_stack, string_stack) or a single character stack.
    │   │     On digit: accumulate k. On [: push current string and k onto stacks,
    │   │     reset. On ]: pop string and k, append k * current_string to the popped
    │   │     prefix
    │   ├── Compare Version Numbers [M]
    │   │   → v1 = list(map(int, version1.split('.'))). Pad shorter list with
    │   │     zeros. Compare element by element
    │   └── Integer to English Words [M]
    │       → Process groups of 3 digits from largest to smallest. For each
    │         non-zero group, call helper(group) and append the appropriate suffix.
    │         Special-case 0
    ├── Pattern Matching
    │   ├── Implement strStr (KMP) [M]
    │   │   → Build LPS: two pointers len_ = 0, i = 1. If needle[i] ==
    │   │     needle[len_], lps[i] = len_ + 1; i++; len_++. Else if len_ > 0, len_ =
    │   │     lps[len_-1]. Else lps[i] = 0; i++. Search: advance i (haystack), j
    │   │     (needle); on mismatch use j = lps[j-1]; when j == len(needle) record
    │   │     match
    │   ├── Repeated Substring Pattern [M]
    │   │   → return s in (s + s)[1:-1]. Alternatively, use KMP: build LPS array
    │   │     for s; if lps[-1] > 0 and len(s) % (len(s) - lps[-1]) == 0, the pattern
    │   │     length is len(s) - lps[-1]
    │   └── Longest Happy Prefix [M]
    │       → Standard LPS build (same as in Implement strStr). Return s[:lps[-1]]
    └── String Manipulation
```

```
    ├── String Compression [M]
    │   → Maintain read and write pointers. For each run, count length and emit
    │     the character plus its decimal digits
    ├── Zigzag Conversion (LC 6) [M]
    │   → Track current_row (starts at 0) and direction (+1 going down, -1
    │     going up). Flip direction when current_row == 0 or current_row ==
    │     numRows - 1. Final answer = concatenation of all row strings
    ├── Valid Palindrome II (LC 680) [M]
    │   → Helper is_palindrome(l, r) checks s[l..r]. Main: walk until mismatch,
    │     then return is_palindrome(left+1, right) or is_palindrome(left,
    │     right-1)
    └── Reverse Words in a String (LC 151) [M]
        → words = s.split() → words.reverse() → return ' '.join(words)
```

tree.md

```
coding/data-structures/tree.md
└── tree.md
    ├── DFS Traversal
    │   ├── Binary Tree Inorder Traversal [M]
    │   │   → while curr or stack: inner while curr pushes all lefts; curr =
    │   │     stack.pop() processes node, appends value; curr = curr.right to explore
    │   │     right subtree
    │   ├── Invert Binary Tree [M]
    │   │   → Post-order recursive swap: Base case not root → None. Recurse left
    │   │     and right, then swap: root.left, root.right = invertTree(root.right),
    │   │     invertTree(root.left). Pre-order also works since swapping is an O(1)
    │   │     local operation
    │   ├── Symmetric Tree [M]
    │   │   → Recursive mirror(l, r) — outer and inner pair matching: Base cases:
    │   │     both None → True (symmetric absence), exactly one None → False.
    │   │     Otherwise: l.val == r.val and mirror(l.left, r.right) and
    │   │     mirror(l.right, r.left)
    │   ├── Maximum Depth of Binary Tree [M]
    │   │   → Post-order recursive max height: Base case not root → 0; otherwise 1
    │   │     + max(maxDepth(left), maxDepth(right))
    │   ├── Minimum Depth of Binary Tree [M]
    │   │   → Base case: not root → 0. Then check left/right nullity before min
    │   ├── Path Sum [M]
    │   │   → DFS subtracting current value — check at leaf: The leaf check is
    │   │     critical — only return True at nodes where not left and not right (both
    │   │     children null), not at any node where partial sum matches
    │   ├── Path Sum II [M]
    │   │   → At leaf (not left and not right) and remaining == 0:
    │   │     result.append(list(path)) (copy! not reference). Then path.pop() on
    │   │     return regardless of whether this was a leaf
    │   ├── Sum Root to Leaf Numbers [M]
    │   │   → dfs(node, curr_num) — curr_num = curr_num * 10 + node.val; at leaf
    │   │     return curr_num; otherwise return dfs(left, curr_num) + dfs(right,
    │   │     curr_num)
    │   ├── Count Complete Tree Nodes [M]
    │   │   → height(node) follows .left pointers only. If left_h == right_h,
    │   │     return 2^left_h + countNodes(root.right); else return 2^right_h +
    │   │     countNodes(root.left)
    │   ├── Count Good Nodes in Binary Tree [M]
    │   │   → DFS with propagated path_max: Update path_max = max(path_max,
    │   │     node.val) before recursing; root is always good
    │   ├── Same Tree [M]
    │   │   → Base cases: both None → True; exactly one None → False; p.val !=
    │   │     q.val → False. Recurse: isSameTree(p.left, q.left) and
    │   │     isSameTree(p.right, q.right)
    │   ├── Subtree of Another Tree [M]
    │   │   → isSubtree(root, subRoot) — if not root: False; if isSameTree(root,
    │   │     subRoot): True; else recurse left and right
    │   ├── Flatten Binary Tree to Linked List [M]
    │   │   → Morris-style in-place threading — find inorder predecessor: For each
    │   │     curr with a left child: find rightmost node in left subtree (prev),
    │   │     wire prev.right = curr.right, move curr.right = curr.left, null
    │   │     curr.left. Advance curr = curr.right
    │   └── Step-By-Step Directions From a Binary Tree Node t... [M]
    │       → DFS to root→start and root→dest paths, strip common prefix: DFS to
    │         record L/R path from root to each target node, then strip common prefix
    │
```

```
├── Level Order BFS
│   ├── Binary Tree Level Order Traversal [M]
│   │   → BFS with level-size snapshot: Inner loop runs exactly level_size
│   │     times; enqueue children during inner loop; append collected level after
│   │     inner loop
│   ├── Binary Tree Zigzag Level Order Traversal [M]
│   │   → left_to_right = True initially; after collecting each level: if not
│   │     left_to_right, level.reverse(); then left_to_right = not left_to_right
│   ├── Binary Tree Right Side View [M]
│   │   → BFS — capture last node per level: Snapshot level size; run inner
│   │     loop; append value only when i == level_size - 1
│   ├── Populating Next Right Pointers in Each Node [M]
│   │   → O(1) space — walk current level to connect next level: Start with
│   │     leftmost = root; inner loop walks the current level using head.next;
│   │     outer loop descends via leftmost = leftmost.left
│   ├── Vertical Order Traversal of a Binary Tree [M]
│   │   → DFS collect (col, row, val) tuples, sort, group: DFS assigning (row,
│   │     col) to each node, collect all tuples, sort, then group by column
│   └── All Nodes Distance K in Binary Tree [M]
│       → Build undirected graph, BFS k steps from target: Build graph first,
│         BFS second, collect nodes at distance == k
├── Lowest Common Ancestor
│   ├── Lowest Common Ancestor of a Binary Tree [M]
│   │   → Post-order recursion — converge at split node: if left and right:
│   │     return root; return left or right
│   └── Lowest Common Ancestor of a BST [M]
│       → Iterative BST-guided descent — O(h) instead of O(n): If both p, q <
│         root, go left. If both > root, go right. Otherwise return root
├── Tree DP
│   ├── Diameter of Binary Tree [M]
│   │   → Post-order height DFS with global diameter update: Return 1 +
│   │     max(left, right) upward; update best[0] with l + r at each node
│   ├── Binary Tree Maximum Path Sum [H]
│   │   → Post-order gain DFS — clamp negatives to 0: gain(node) = node.val +
│   │     max(l, r) returned upward; global update = node.val + l + r (where l
│   │     and r already have negatives clamped to 0)
│   ├── House Robber III [M]
│   │   → Post-order DP returning (rob, skip) pair: rob = node.val + l_skip +
│   │     r_skip; skip = max(l_rob, l_skip) + max(r_rob, r_skip). Post-order:
│   │     compute children first
│   ├── Binary Tree Cameras [M]
│   │   → Greedy post-order — delay cameras upward: Null nodes return 1
│   │     (trivially covered). If any child returns 0, place a camera here
│   │     (return 2, increment count). If any child has a camera (returns 2),
│   │     this node is covered (return 1). Otherwise return 0 (push
│   │     responsibility to parent)
│   └── Path Sum III [M]
│       → DFS with prefix sum hash map + backtracking: DFS — add node.val to
│         running sum, query map for running_sum - targetSum, increment map,
│         recurse children, then decrement map on backtrack (critical: prevents
│         prefix sum from leaking into sibling branches)
├── BST Operations
│   ├── Validate Binary Search Tree [M]
│   │   → Recursive range validation — propagate (lo, hi) bounds:
│   │     validate(node, lo, hi) — fail if not lo < node.val < hi, else recurse
│   │     with tightened bounds
│   ├── Kth Smallest Element in a BST [M]
│   │   → Iterative inorder with early exit at k: while stack or root: push all
│   │     left children, pop, decrement k, if k == 0 return val, else advance to
│   │     right child
│   ├── Insert and Delete in BST [M]
│   │   → Delete's two-child case: find inorder successor (leftmost in right
│   │     subtree), copy its value to current node, then delete successor from
│   │     right subtree
│   ├── Range Sum of BST [M]
│   │   → DFS with BST pruning — skip entire subtrees: Only recurse left if
│   │     node.val > low; only recurse right if node.val < high
│   └── Recover Binary Search Tree [M]
│       → Inorder DFS — detect inversion pair(s), swap values: One or two
│         inversion sites. Always: first = prev at first inversion; second = curr
│         at each inversion (covers both one and two inversion cases)
├── Construction / Serialization
│   ├── Serialize and Deserialize Binary Tree [H]
│   │   → Preorder DFS with null markers — iterator-based deserialization:
│   │     Serialize: DFS pre-order appending values or #. Deserialize: iterate
│   │     tokens; # → return None; otherwise create node, recurse left, recurse
│   │     right. Use an iterator to advance position across recursive calls
│   ├── Construct Binary Tree from Preorder and Inorder T... [M]
│   │   → Preorder index advance + inorder hash map for O(1) root lookup:
│   │     build(in_left, in_right) — take preorder[pre_idx] as root, find its
│   │     inorder position mid, build left subtree with in_left..mid-1, right
│   │     subtree with mid+1..in_right
│   └── Construct Binary Tree from Inorder and Postorder ... [M]
│       → build(in_left, in_right) — take postorder[post_idx] as root,
│         decrement index, find root in inorder hash map as mid, build right
│         subtree (mid+1, in_right) FIRST, then left (in_left, mid-1)
└── Special Tree Problems
    ├── Count Complete Tree Nodes (LC 222) [M]
    │   → left_height = length of left spine. right_height = length of right
    │     spine. If equal, return (1 << left_height) - 1. Else return 1 +
    │     count(root.left) + count(root.right)
    ├── Binary Tree Cameras (LC 968) [H]
    │   → For each node: if either child is uncovered (state 0), place a camera
    │     here (state 1, increment count). If either child has a camera (state
    │     1), this node is covered (state 2). Otherwise (both children covered
    │     without cameras), return state 0 — let the parent handle coverage.
    │     After DFS, if root returns state 0, place one more camera
    └── Recover Binary Search Tree (LC 99) [M]
        → After traversal, swap first_bad.val and second_bad.val
```

trie.md

```
coding/data-structures/trie.md
└── trie.md
    ├── Core Trie Implementation
    │   └── Implement Trie (Prefix Tree) [M]
    │       → EDGE CASES:: - insert: walk the tree character by character, creating
    │         nodes as needed, set is_end = True at the last character. - search:
    │         walk the tree; return False if any character is missing; return
    │         node.is_end at the end — this distinguishes "apple" (exact) from "app"
    │         (only prefix). - startsWith: same walk as search but return True after
    │         the walk completes regardless of is_end. - EDGE CASES: The empty string
    │         is valid only if you explicitly mark the root as terminal; duplicate
    │         inserts are idempotent with a boolean trie; deletes must prune only
    │         dead branches so shared prefixes stay intact
    ├── Autocomplete / Prefix Search
    │   ├── Design Search Autocomplete System [M]
    │   │   → On insert(sentence, freq): walk each character; at each node update
    │   │     node.counts[sentence] += freq. On input(c): if '#', save the current
    │   │     input with frequency 1 (updating all ancestor nodes), reset state.
    │   │     Otherwise, advance curr_node by one character and return the top 3
    │   │     sentences from curr_node.counts, ordered by (-frequency, sentence)
    │   ├── Map Sum Pairs [M]
    │   │   → On insert(key, val): if key already exists, compute delta = val -
    │   │     old_val to avoid double-counting. Walk each character of key; at each
    │   │     node add delta to node.prefix_sum. On sum(prefix): walk to the prefix
    │   │     node and return node.prefix_sum
    │   ├── Longest Word in Dictionary [M]
    │   │   → Insert all words. BFS from root — only enqueue child nodes where
    │   │     is_end = True. At each node, track the current word. Update the answer
    │   │     when a longer (or lexicographically smaller tie) word is found
    │   └── Replace Words [M]
    │       → Insert all roots into the trie. For each word in the sentence, walk
    │         the trie; if is_end is reached at depth d, replace the word with
    │         word[:d]. If the walk exits without finding a root, keep the original
    │         word
    ├── Trie + Backtracking
    │   ├── Word Search II [H]
    │   │   → Build trie, storing the actual word string at is_end nodes. DFS: if
    │   │     ch not in node.children, return immediately. If node.word is set,
    │   │     record and clear it (deduplication). Mark cell as '#' during DFS;
    │   │     restore on backtrack. After DFS, prune dead trie nodes (del
    │   │     node.children[ch] when subtree is empty) to avoid re-exploring
    │   │     exhausted branches
    │   └── Word Squares [M]
    │       → Build trie; at each node store all words whose prefix passes through
    │         it (node.words). Backtrack row by row: at row k, the required prefix =
    │         square[0][k] + square[1][k] + ... + square[k-1][k] (k-th column of
    │         already-placed words). Look up all words with that prefix in the trie.
    │         Try each as square[k]; recurse to row k+1. Base case: len(square) ==
    │         word_length → record result
    ├── XOR Trie
    │   └── Maximum XOR of Two Numbers in an Array [M]
    │       → insert(num): for bits 31 → 0, compute b = (num >> bit) & 1, follow or
    │         create the b branch. max_xor(num): for each bit, try to go to 1 - b
    │         (opposite = XOR bit = 1); if that branch exists, take it and add 1 <<
    │         bit to the XOR; otherwise take the b branch. Build the trie with all
    │         numbers, then query each number for its best XOR pair
    ├── Miscellaneous
    │   └── Number of Distinct Substrings in a String [M]
    │       → Start with an empty trie. For each suffix, insert it — count the
    │         number of new nodes created. The total count is the number of distinct
    │         substrings. (Equivalently: total nodes in the trie minus the root.)
    ├── Core Trie Operations (Extended)
    │   └── Add and Search Word [M]
    │       → addWord: standard trie insert. search(word, node, i): if i ==
    │         len(word) return node.is_end. If word[i] == '.', recurse into every
    │         child and return True if any succeeds. Otherwise follow the exact child
    │         as usual
    ├── Prefix Problems
    │   ├── Search Suggestions System [M]
    │   │   → Sort products. Insert each: at every node on the path, append the
    │   │     word to node.suggestions if len < 3. Query: walk the search word prefix
    │   │     character by character; at each step return node.suggestions (or [] if
    │   │     the branch doesn't exist, and stay dead for subsequent characters)
    │   ├── Prefix and Suffix Search [M]
    │   │   → For each word words[i], insert every suffix of that word followed by
    │   │     '#' and the full word. Query f(pref, suff) walks suff + '#' + pref in
    │   │     the trie and returns the stored max index, or -1 if the path is missing
    │   ├── Count Words With Given Prefix [M]
    │   │   → Insert each word, incrementing node.count at every node on the path.
    │   │     Query: walk pref; if the walk succeeds, return node.count. If any
    │   │     character is missing, return 0
    │   └── Implement Magic Dictionary [M]
    │       → _dfs(node, word, i, diff): base case i == len(word) → return
    │         node.is_end and diff == 1. At each step: for exact char, recurse with
    │         same diff. For all other children, recurse with diff + 1 (only if diff
    │         == 0). Return True if any branch returns True
    ├── XOR Trie (Extended)
    │   ├── Maximum XOR With an Element From Array [M]
    │   │   → Sort nums. Sort queries by mi (keep original index for output). Two
    │   │     pointers: pointer j into nums. For each query (xi, mi), advance j while
    │   │     nums[j] <= mi, inserting into trie. If trie is empty (no num ≤ mi),
    │   │     answer is -1. Otherwise query max_xor(xi)
    │   └── Count Pairs With XOR in a Range [M]
    │       → count_leq(num, limit): walk bit by bit from MSB. At each bit b of num
    │         and l of limit: if l == 1, all numbers with XOR bit 0 at this position
    │         contribute (they have XOR < current prefix) — add node.children[b].size
    │         if it exists, then continue down the 1-b branch to keep XOR bit = 1. If
    │         l == 0, must go down b branch (XOR bit = 0)
    └── Bitwise Trie / Other
        ├── Design File System [M]
        │   → createPath(path, value): split path by /, ignore leading empty
        │     string. Walk all but the last component — if any is missing, return
        │     False. At the last component, if the node already exists and was
        │     explicitly created (value != -1), return False; else create it with the
        │     value. get(path): walk all components; return node's value or -1 if
        │     path doesn't exist
        ├── Stream of Characters [M]
        │   → Build trie of reversed words. Keep active: list[TrieNode]. On
        │     query(c): new_active = []; for each node in active, if c in
        │     node.children, add child to new_active. Also check root for c (new
        │     suffix starting). If any node in new_active has is_end = True, return
        │     True
        ├── Palindrome Pairs [M]
        │   → At each trie node, store a list of word indices whose reversed word
```

```
│   │   │   ends here but the trie path continues — these are "prefix palindrome"
│   │   │   candidates. Walk trie with w; if node.word_idx != -1 and the remaining
│   │   │   w[i:] is a palindrome, record (word_idx, i_word). After exhausting w,
│   │   │   collect from node.palindrome_suffixes (stored during build)
│   │   └── Design Search Autocomplete System (Trie + DFS Var... [M]
│   │       → insert(sentence, freq): standard trie, set node.freq = freq at
│   │         terminal. query(prefix): walk to prefix node. DFS collecting (freq,
│   │         built_string) for all is_end nodes. Sort and return top 3
│   ├── Suffix Trie / Advanced
│   │   ├── Implement Trie II (Count Operations) [M]
│   │   │   → insert: walk, incrementing node.pass_count at every node,
│   │   │     node.end_count at terminal. countWordsStartingWith(prefix): walk to
│   │   │     prefix node, return node.pass_count. countWordsEqualTo(word): walk to
│   │   │     terminal, return node.end_count. erase(word): walk, decrementing
│   │   │     node.pass_count; decrement node.end_count at terminal. Optionally prune
│   │   │     nodes where pass_count == 0
│   │   ├── Shortest Unique Prefix for Every Word [M]
│   │   │   → Insert all words, incrementing node.pass_count at each node. For each
│   │   │     word, walk character by character; the first node with pass_count == 1
│   │   │     marks the end of the shortest unique prefix
│   │   └── Maximum XOR of Two Numbers — Prefix Hash Approach [M]
│   │       → max_xor = 0. For bit b from 31 to 0: mask = max_xor | (1 << b).
│   │         Compute prefix set {num & mask for num in nums}. Try candidate =
│   │         max_xor | (1 << b). If any two prefixes a, b in the set satisfy a ^ b
│   │         == candidate (i.e., candidate ^ a is in the set), then max_xor =
│   │         candidate. Else leave max_xor unchanged (this bit stays 0)
│   └── Trie Applications
│       ├── Replace Words (LC 648) [M]
│       │   → Insert all roots. For each sentence word, traverse the trie; if a
│       │     node has is_end = True, return the prefix built so far. If traversal
│       │     ends without a match, keep the original word
│       ├── Palindrome Pairs (LC 336) [M]
│       │   → For each word at index i, split at every position k (0 to len): if
│       │     word[:k] is a palindrome and reverse(word[k:]) is in the map (and not
│       │     i), add (map[reverse], i). If word[k:] is a palindrome and
│       │     reverse(word[:k]) is in the map, add (i, map[reverse]). Avoid
│       │     duplicates by only doing suffix-palindrome check for k > 0
│       └── Word Squares (LC 425) [M]
│           → Insert all words into the trie, at each node also storing the list of
│             words with that prefix. Backtrack with build(step, square): if step ==
│             n, save the square. Otherwise extract prefix from the current partial
│             square, look up matching words in the trie, and recurse
```

coding/algorithms/

backtracking.md

```
coding/algorithms/backtracking.md
└── backtracking.md
    ├── Subsets / Combinations
    │   ├── Subsets (Power Set) [M]
    │   │   → Record subset at every node (not just leaves) — each partial path is
    │   │     itself a valid subset
    │   ├── Subsets II (with duplicates) [M]
    │   │   → Same structure as Subsets but with if i > start and nums[i] ==
    │   │     nums[i-1]: continue
    │   ├── Combination Sum (unbounded) [M]
    │   │   → Recurse with bt(i, remaining - candidates[i]) — same i, not i+1
    │   ├── Combination Sum II (0/1 — no reuse) [M]
    │   │   → if i > start and candidates[i] == candidates[i-1]: continue
    │   ├── Combinations [M]
    │   │   → Pruning: if n - i + 1 < k - len(path): break — not enough numbers
    │   │     remain
    │   └── Letter Combinations of a Phone Number [M]
    │       → Base case: d == len(digits) → record. No pruning needed — every path
    │         is valid
    ├── Permutations
    │   ├── Permutations [M]
    │   │   → Base case: len(path) == n. No pruning — every branch leads to a valid
    │   │     permutation
    │   ├── Permutations II (with duplicates) [M]
    │   │   → if i > 0 and nums[i] == nums[i-1] and not used[i-1]: continue
    │   ├── Next Permutation (iterative approach) [M]
    │   │   → If no pivot found (fully descending), entire array is reversed
    │   └── Letter Case Permutation [M]
    │       → String building via list (mutable); convert at leaf
    ├── String Backtracking
    │   ├── Generate Parentheses [M]
    │   │   → Leaf count = Catalan number C(n). The invariant close < open ensures
    │   │     every partial string is a valid prefix
    │   ├── Palindrome Partitioning [M]
    │   │   → Precompute is_pal[i][j] via DP in O(n²). Then backtracking is O(2^n)
    │   │     calls × O(1) palindrome check
    │   ├── Remove Invalid Parentheses [H]
    │   │   → Compute open_rem, close_rem first; backtrack with those exact budgets
    │   ├── Word Search [M]
    │   │   → Early return True on complete match. Board is restored on each
    │   │     backtrack
    │   └── Expression Add Operators (LC 282) [H]
    │       → Start at index=0, total=0, last=0. No leading zeros: skip if num_str
    │         starts with '0' and length > 1
    ├── Board / Matrix Backtracking
    │   ├── N-Queens [M]
    │   │   → Add to sets before recursing; remove after. Leaf (row==n) records the
    │   │     board
    │   ├── Sudoku Solver [H]
    │   │   → box_id = (r//3)*3 + c//3. Linear scan for next empty cell at each
    │   │     recursion level
    │   └── Unique Paths III [M]
    │       → Track remaining count of unvisited non-obstacle cells. Prune when
    │         stuck (all 4 neighbors blocked and remaining > 1)
    ├── Trie + Backtracking
    │   └── Word Search II [H]
    │       → Key optimizations: (1) Delete found words from Trie (node.word =
    │         None) to avoid duplicates. (2) Prune empty Trie nodes (del node[ch]
    │         after DFS) — reduces future DFS calls significantly
    ├── Constraint Satisfaction / Pruning-Heavy
    │   ├── Combination Sum III [M]
    │   │   → Because the domain is tiny (1-9), no sorting needed — it's inherently
    │   │     sorted. Upper bound prune: if remaining > sum(range(start,
    │   │     10))[:k-len(path)], stop
    │   ├── Target Sum [M]
    │   │   → Backtracking here: O(2^n). DP reduction: let P = sum of positives, N
    │   │     = sum of negatives. P - N = target, P + N = total → P = (target +
    │   │     total) / 2. Count subsets summing to P
    │   ├── Beautiful Arrangement [M]
    │   │   → Pruning is implicit: invalid choices are simply not tried. Iterate
    │   │     numbers in decreasing order to hit more valid placements early
    │   │     (empirically faster)
    │   └── Restore IP Addresses [M]
    │       → After choosing 4 segments, the entire string must be consumed.
    │         Feasibility check: remaining_digits must be between remaining_segments
    │         and 3 * remaining_segments
    ├── String Backtracking (continued)
    │   ├── Word Break II [M]
    │   │   → Memoization key is start index. Value is list of sentence suffixes
    │   │     from that position. Build full sentences by prepending current word
    │   └── Palindrome Partitioning II (Minimum Cuts) [M]
    │       → WHAT (DP):: dp[0] = 0 (empty prefix needs 0 cuts). Final answer:
    │         dp[n] - 1 (subtracting the artificial initial cut). Equivalently, dp[i]
    │         = min cuts for first i chars → dp[n]
    ├── Grid Traversal
    │   ├── Rat in a Maze [M]
    │   │   → Path encoded as direction string ('D','L','R','U'). Sort output
    │   │     lexicographically — lexicographic DFS order (try D,L,R,U
    │   │     alphabetically) naturally produces sorted output
    │   └── Word Search (All Occurrences) [M]
    │       → Each DFS is independent — board restored fully between starting
    │         cells. Use in-place '#' marking within a single DFS call
    ├── Advanced Backtracking
    │   ├── N-Queens II (Count Only) [M]
    │   │   → Bitmask trick: valid columns = ((1<<n) - 1) & ~(cols | diags |
    │   │     anti_diags). Extract LSB: pos = available & (-available). Iterate:
    │   │     available &= available - 1
    │   ├── Word Break (Decision — Backtracking + Memo) [M]
    │   │   → Check only words in the dictionary (not all prefixes). Short-circuit
    │   │     on first True. BFS or DP are preferred in interviews
    │   └── Generate All Valid IP Addresses (Generalized Segm... [M]
    │       → Feasibility: remaining_chars must be in [k-1, (k-1)*max_seg_len +
    │         max_seg_len] — i.e., between 1 and max_seg_len per remaining segment
    └── Backtracking — Hard Problems
        └── Remove Invalid Parentheses (LC 301) [H]
            → is_valid(s): count open brackets, decrement on ), return false if
              count < 0, return count == 0 at end
```

binary-search.md

```
coding/algorithms/binary-search.md
└── binary-search.md
    ├── Lower Bound / Upper Bound
    │   ├── Search Insert Position (LC 35) [M]
    │   │   → If nums[mid] < target → lo = mid + 1; else hi = mid. Loop terminates
    │   │     at lo == hi
    │   ├── First Bad Version (LC 278) [M]
    │   │   → if isBadVersion(mid): hi = mid (don't discard mid); else lo = mid +
    │   │     1. Terminates at lo == hi
    │   ├── Find Smallest Letter Greater Than Target (LC 744) [M]
    │   │   → If letters[mid] <= target → lo = mid + 1; else hi = mid. Answer is
    │   │     letters[lo % len]
    │   └── H-Index II (LC 275) [M]
    │       → If citations[mid] < n - mid → not enough citations here → lo = mid +
    │         1; else hi = mid
    ├── Binary Search on Answer (Predicate Pattern)
    │   ├── Koko Eating Bananas (LC 875) [M]
    │   │   → ceil(p/k) without math.ceil → -(-p // k). If feasible → hi = mid (try
    │   │     slower); else lo = mid + 1
    │   ├── Capacity To Ship Packages Within D Days (LC 1011) [H]
    │   │   → Greedy feasibility: greedily fill each day; when adding next weight
    │   │     exceeds capacity, start a new day
    │   ├── Split Array Largest Sum (LC 410) [H]
    │   │   → Greedily extend current subarray; when adding next element would
    │   │     exceed max_sum, start new part. If parts ≤ k → feasible
    │   ├── Minimize Max Distance to Gas Station (LC 774) [M]
    │   │   → 100 iterations of binary search achieves precision ~1e-30, well
    │   │     within the required 1e-6
    │   └── Minimum Speed to Arrive on Time (LC 1870) [M]
    │       → Early exit: if hour <= n - 1, impossible (each of n-1 waits costs at
    │         least 1 hour)
    ├── Rotated Sorted Array
    │   ├── Search in Rotated Sorted Array (LC 33) [M]
    │   │   → If nums[lo] <= nums[mid], left half [lo, mid] is sorted. If target in
    │   │     [nums[lo], nums[mid]) → go left; else go right
    │   ├── Find Minimum in Rotated Sorted Array (LC 153) [M]
    │   │   → if nums[mid] > nums[hi]: lo = mid + 1 else hi = mid. Do NOT compare
    │   │     with nums[lo]
    │   ├── Search in Rotated Sorted Array II (LC 81) [M]
    │   │   → On ambiguity → lo += 1; hi -= 1. Worst case O(n) for all-same arrays
    │   └── Find Minimum in Rotated Sorted Array II (LC 154) [M]
    │       → Three-way branch on nums[mid] vs nums[hi]
    ├── Peak Element
    │   ├── Find Peak Element (LC 162) [M]
    │   │   → if nums[mid] < nums[mid+1]: lo = mid + 1 else hi = mid. Terminates at
    │   │     lo == hi which is a peak
    │   ├── Peak Index in a Mountain Array (LC 852) [M]
    │   │   → Same code as LC 162; mountain guarantee makes the result unique
    │   └── Find a Peak Element in a 2D Grid (LC 1901) [M]
    │       → If mat[max_row][mid_col] < mat[max_row][mid_col+1] → a peak exists to
    │         the right; else left or at mid
    ├── Median / Order Statistics
    │   ├── Median of Two Sorted Arrays (LC 4) [H]
    │   │   → If max_left1 > min_right2 → partition too far right in nums1 → hi = i
    │   │     - 1; else lo = i + 1
    │   ├── Kth Smallest Element in a Sorted Matrix (LC 378) [M]
    │   │   → Start top-right. If matrix[row][col] <= d → all col+1 elements in
    │   │     this row qualify → row += 1; else col -= 1. If count ≥ k → hi = mid;
    │   │     else lo = mid + 1. Answer is always an actual matrix element
    │   └── Find K-th Smallest Pair Distance (LC 719) [M]
    │       → lo = 0, hi = nums[-1] - nums[0]. Count pairs with distance ≤ mid:
    │         two-pointer left tracking the start of the window for each right.
    │         Return the smallest mid where count >= k
    ├── 2D Matrix Binary Search
    │   ├── Search a 2D Matrix (LC 74) [M]
    │   │   → lo=0, hi=m*n-1. While lo<=hi: mid=(lo+hi)//2;
    │   │     val=matrix[mid//n][mid%n]. Compare with target, standard binary search
    │   │     logic
    │   └── Search a 2D Matrix II (LC 240) [M]
    │       → Each step eliminates a row or column. O(m+n) total
    ├── Classic / Miscellaneous
    │   ├── Guess Number Higher or Lower (LC 374) [M]
    │   │   → lo=1, hi=n. While lo <= hi: if guess(mid)==0 return mid; if
    │   │     guess(mid)==-1 the answer is lower → hi=mid-1; else lo=mid+1
    │   ├── Find First and Last Position (LC 34) [M]
    │   │   → First = lower_bound(target). If nums[first] != target → not found.
    │   │     Last = upper_bound(target) - 1
    │   ├── Minimum Number of Days to Make m Bouquets (LC 1482) [M]
    │   │   → Count bouquets formed; if >= m → feasible. Early exit: if m * k >
    │   │     len(bloomDay) → impossible
    │   ├── Find K Closest Elements (LC 658) [M]
    │   │   → If x - arr[mid] > arr[mid+k] - x → window is too far left → lo = mid
    │   │     + 1; else hi = mid. Answer is arr[lo : lo + k]
    │   └── Count of Smaller Numbers After Self (LC 315) [M]
    │       → Insert nums[i] into the sorted list (using insort). Count =
    │         bisect_left(sorted_list, nums[i])
    └── Second Occurrence / Exact Match Variants
        ├── Search a 2D Matrix — Row + Column BS (LC 74 varia... [M]
        │   → Row search: if matrix[mid][0] <= target: lo = mid else hi = mid - 1.
        │     Then standard column search
        ├── Sqrt(x) — Integer Square Root (LC 69) [M]
        │   → lo = 0, hi = x. While lo <= hi: mid = (lo + hi) // 2. If mid * mid <=
        │     x set result = mid and lo = mid + 1. Else hi = mid - 1
        ├── Find the Duplicate Number (LC 287) — BS on Value [M]
        │   → if count > mid: hi = mid else lo = mid + 1. Not a standard in-place
        │     BS — it's BS on the value space, not the index space
        └── Longest Increasing Subsequence — Length via BS (L... [M]
            → If num > tails[-1] → append (extend LIS). Else → replace tails[pos] =
              num (maintain smallest tails for future options)
```

bit-manipulation.md

```
coding/algorithms/bit-manipulation.md
└── bit-manipulation.md
    ├── XOR Properties
    │   ├── Single Number [M]
    │   │   → reduce(xor, nums). One pass, no extra memory
    │   ├── Single Number II (Three Copies — Bit Counting) [M]
    │   │   → Iterate bit positions 0–31. Sum how many numbers have bit i set.
    │   │     count % 3 gives bit i of the unique number
    │   ├── Single Number III (Two Unique — XOR Split) [M]
    │   │   → Two-pass: pass 1 computes xor_sum; pass 2 partitions and XORs each
    │   │     group
    │   ├── Missing Number [M]
    │   │   → reduce(xor, range(n+1)) ^ reduce(xor, nums)
    │   └── Find the Difference [M]
    │       → Convert chars to ord values; XOR all together
    ├── Bit Counting
    │   ├── Number of 1 Bits (Hamming Weight) [M]
    │   │   → Count iterations until n == 0
    │   ├── Hamming Distance [M]
    │   │   → XOR then apply Brian Kernighan's or bin().count('1')
    │   ├── Counting Bits (DP Approach) [M]
    │   │   → Single pass i = 1 to n; use previously computed values
    │   └── Reverse Bits [M]
    │       → 32 iterations. After 32 iterations, undo the final left shift (or
    │         structure the loop to avoid it)
    ├── Bit Tricks
    │   ├── Power of Two [M]
    │   │   → Guard n > 0 is mandatory — 0 & (0-1) == 0 but 0 is not a power of two
    │   ├── Power of Four [M]
    │   │   → n > 0 and (n & (n-1)) == 0 and (n & 0x55555555) != 0
    │   ├── Bitwise AND of Numbers Range [M]
    │   │   → Count shifts while left != right; shift both right; shift result back
    │   │     left
    │   └── Sum of Two Integers Without + or - (LC 371) [M]
    │       → While b != 0: carry = (a & b) << 1; a = a ^ b; b = carry. Return a.
    │         In Python, need to handle 32-bit overflow with masking: mask =
    │         0xFFFFFFFF; work modulo mask; at end if a > 0x7FFFFFFF: a = ~(a ^ mask)
    ├── Bitmask DP
    │   ├── Subsets via Bitmask [M]
    │   │   → For each mask, check each bit position k; include nums[k] if mask &
    │   │     (1 << k)
    │   ├── Shortest Path Visiting All Nodes (BFS + Bitmask) [M]
    │   │   → Multi-source BFS — start from all nodes simultaneously (each with its
    │   │     own bit set). State space: O(N · 2^N). BFS guarantees minimum steps
    │   ├── Smallest Sufficient Team (Bitmask DP) [M]
    │   │   → Initialize dp[0] = []. For each existing state mask and each person,
    │   │     compute new coverage. Return dp[(1<<m)-1]
    │   └── Travelling Salesman Problem (TSP) — N ≤ 20 (LC-st... [M]
    │       → - State: mask (bitmask of visited cities), i (current city). - Base:
    │         dp[1<<0][0] = 0 (start at city 0, only city 0 visited). - Transition:
    │         for each city j not in mask: dp[mask|(1<<j)][j] =
    │         min(dp[mask|(1<<j)][j], dp[mask][i] + cost[i][j]). - Answer: min over
    │         all i of dp[(1<<n)-1][i] + cost[i][0]
    ├── XOR Trie
    │   ├── Maximum XOR of Two Numbers in an Array [M]
    │   │   → Trie node has children {0: ..., 1: ...}. Insert: bit-by-bit from bit
    │   │     31 to 0. Query: at each level, try to go to 1 - bit; if present, add 1
    │   │     << bit_pos to result
    │   └── Maximum XOR with Element from Array (LC 1707) [M]
    │       → 1. Sort nums. Sort queries with original indices by mi. 2. Build XOR
    │         Trie supporting insert and max_xor_query. 3. For each query (xi, mi,
    │         original_idx): insert all nums ≤ mi into Trie. Query Trie for max XOR
    │         with xi. If Trie empty, answer is -1
    ├── Bitmask State Space Search
    │   └── Shortest Path with Keys and Locks [M]
    │       → Preprocess grid for start position, key count. BFS with state space
    │         O(R · C · 2^K). Goal: keys_mask == (1 << num_keys) - 1
    └── Encoding / Decoding with Bits
        ├── UTF-8 Validation [M]
        │   → Iterate bytes. If expected_continuations > 0, check byte & 0xC0 ==
        │     0x80. Otherwise classify the byte's prefix to set new expected count.
        │     Invalid if counts mismatch or byte is out of range
        ├── Gray Code [M]
        │   → For i and i+1, (i ^ (i>>1)) ^ ((i+1) ^ ((i+1)>>1)) always has exactly
        │     one bit set (the carry bit). This is provable by induction on the
        │     binary addition carry chain
        ├── Decode XORed Array [M]
        │   → Initialize arr = [first]. For each value in encoded, append
        │     encoded[i] ^ arr[-1]
        └── Find the Duplicate Number (Bit Approach) [M]
            → Outer loop over 32 bit positions; inner loop counts set bits in nums
              and in range(1, n+1). If count_nums > count_range, set that bit in the
              answer
```

divide-and-conquer.md

```
coding/algorithms/divide-and-conquer.md
└── divide-and-conquer.md
    ├── Classic D&C
    │   ├── Merge Two Sorted Lists [M]
    │   │   → Base: if either list is None, return the other. Compare l1.val vs
    │   │     l2.val; attach smaller, recurse
    │   ├── Merge K Sorted Lists [H]
    │   │   → While len(lists) > 1: pop two, merge them, push result back. Or
    │   │     recurse: merge(lists[:mid]) + merge(lists[mid:]) as left/right
    │   ├── Median of Two Sorted Arrays [H]
    │   │   → Ensure A is shorter. Binary search lo to hi (size of A). At midpoint
    │   │     i, compute j. If A[i-1] > B[j], go left; if B[j-1] > A[i], go right;
    │   │     else compute median from boundary values
    │   └── Pow(x, n) (Fast Exponentiation) [M]
    │       → Handle n < 0 by inverting x and negating n. Iterative with bit
    │         manipulation is cleaner than recursive
    ├── QuickSelect
    │   ├── Kth Largest Element in an Array [M]
    │   │   → Randomize pivot to avoid O(n²) worst case on sorted input. kth
    │   │     largest = (n-k)th index in 0-indexed sorted array
    │   ├── K Closest Points to Origin [M]
    │   │   → 2 + p[1]: Same partition logic; compare by dist(p) = p[0]2 + p[1]2.
    │   │     After QuickSelect, points[:k] are the answer
    │   └── Top K Frequent Elements [M]
    │       → freq = Counter(nums). Run QuickSelect on list(freq.keys()) comparing
    │         by freq[key]. Target index = n - k
    ├── Merge Sort Variants (D&C with Counting)
    │   ├── Count of Smaller Numbers After Self [M]
    │   │   → Track right_picked count during merge. Each time a right element is
    │   │     chosen over remaining left[i:] elements, increment
    │   │     counts[left[i].index]
    │   ├── Reverse Pairs [M]
    │   │   → Two-pointer count: for each left[i], advance j while left[i] > 2 *
    │   │     right[j]; add j to total. Then perform standard merge
    │   ├── Count of Range Sum [M]
    │   │   → For each element in left half L[i], find window [lo, hi) in right
    │   │     half where lower ≤ R[k] - L[i] ≤ upper. Two pointers maintain this
    │   │     window as i advances
    │   ├── Majority Element (D&C Approach) [M]
    │   │   → Base: single element is its own majority. Count occurrences of left
    │   │     and right candidates in the full subarray; return whichever exceeds
    │   │     half
    │   └── Majority Element II [M]
    │       → Maintain two (candidate, count) pairs. On each element: if matches
    │         candidate, increment; else decrement both; if count reaches 0, replace.
    │         Final verification pass confirms actual frequency
    ├── Matrix / 2D D&C
    │   ├── Super Pow (Modular Exponentiation) [M]
    │   │   → Base case: b is empty → return 1. At each step, multiply pow(pow(a,
    │   │     b[:-1]), 10, mod) * pow(a, b[-1], mod)
    │   └── Matrix Exponentiation (Fibonacci in O(log n)) [M]
    │       → Same repeated-squaring loop as scalar fast pow; replace scalar mul
    │         with matrix mul
    ├── Geometric D&C
    │   └── Closest Pair of Points [M]
    │       → Base case: n ≤ 3, use brute force
    ├── Binary Search D&C
    │   ├── Search in Rotated Sorted Array [M]
    │   │   → If nums[lo] <= nums[mid], left half is sorted. If nums[lo] <= target
    │   │     < nums[mid], go left; else go right. Mirror logic when right half is
    │   │     sorted
    │   └── Find Minimum in Rotated Sorted Array [M]
    │       → Compare nums[mid] with nums[hi]. If nums[mid] > nums[hi], lo = mid +
    │         1. Else hi = mid. Converges when lo == hi
    ├── Tree Construction D&C
    │   └── Construct Binary Tree from Preorder and Inorder [M]
    │       → Precompute an index map of {value: inorder_index} for O(1) lookup.
    │         Track preorder start offset instead of slicing to stay O(n) total
    ├── Maximum Subarray D&C
    │   └── Maximum Subarray (Divide and Conquer) [E]
    │       → Cross sum: scan left from mid accumulating suffix max; scan right
    │         from mid+1 accumulating prefix max; sum them
    ├── Expression D&C
    │   ├── Different Ways to Add Parentheses [M]
    │   │   → Base case: string is a number → return [int(string)]. Memoize on (lo,
    │   │     hi) or the sub-string to avoid recomputing overlapping sub-expressions
    │   └── Expression Add Operators [M]
    │       → At each position, try every prefix length as the next number. Append
    │         +, -, *, or nothing (first number). For *: curr_val = curr_val - last +
    │         last * num; last = last * num
    ├── QuickSelect Variants
    │   └── Kth Smallest Element in a Sorted Matrix [M]
    │       → Staircase count: start at top-right corner; if matrix[r][c] <= mid,
    │         add r+1 (all rows above in column c are ≤ mid), move right; else move
    │         up. O(n) per count step
    └── Divide and Conquer — More Problems
        └── Different Ways to Add Parentheses (LC 241) [M]
            → Memoize with a dict keyed on the expression string to avoid
              recomputing the same subexpression
```

dynamic-programming.md

```
coding/algorithms/dynamic-programming.md
└── dynamic-programming.md
    ├── Linear DP (1-D)
    │   ├── Climbing Stairs [E]
    │   │   → dp[i] = dp[i-1] + dp[i-2]; base dp[0]=1, dp[1]=1. This is Fibonacci.
    │   │     Space collapses to two variables
    │   ├── Min Cost Climbing Stairs [E]
    │   │   → dp[i] = cost[i] + min(dp[i-1], dp[i-2]); answer = min(dp[n-1],
    │   │     dp[n-2])
    │   ├── House Robber [M]
    │   │   → dp[i] = max(dp[i-1], dp[i-2] + nums[i]); base dp[0]=nums[0],
    │   │     dp[1]=max(nums[0],nums[1])
    │   ├── House Robber II (Circular) [M]
    │   │   → max(rob(0..n-2), rob(1..n-1)) — run linear House Robber twice
    │   ├── Maximum Subarray (Kadane's — DP view) [E]
    │   │   → dp[i] = max(nums[i], dp[i-1] + nums[i]); answer = max(dp). Collapses
    │   │     to one variable
    │   ├── Word Break [M]
    │   │   → dp[i] = any(dp[j] and s[j:i] in word_set) for j in [i-max_len, i).
    │   │     Base: dp[0]=True
    │   └── Decode Ways [M]
    │       → add dp[i-1] if s[i-1] != '0'; add dp[i-2] if 10 ≤ s[i-2:i] ≤ 26.
    │         Base: dp[0]=1
    ├── 0/1 Knapsack
    │   ├── Subset Sum Problem [M]
    │   │   → iterate items; for each item x, sweep j from T down to x: dp[j] |=
    │   │     dp[j - x]. Reverse sweep prevents reuse of same item (0/1 knapsack).
    │   │     Base: dp[0]=True
    │   ├── Partition Equal Subset Sum [M]
    │   │   → Odd total → return False immediately. Then run 0/1 knapsack DP
    │   ├── Target Sum [M]
    │   │   → dp[j] += dp[j - x] in reverse order (0/1 knapsack). Base: dp[0]=1
    │   └── Last Stone Weight II [M]
    │       → Standard 0/1 knapsack boolean DP; answer = total - 2 ×
    │         max_j_where_dp[j]
    ├── Unbounded Knapsack
    │   ├── Coin Change (Min Coins) [M]
    │   │   → dp[i] = min(dp[i-c] + 1) for each coin c ≤ i; forward sweep (reuse
    │   │     allowed). Base: dp[0]=0, rest inf
    │   ├── Coin Change II (Total Ways) [M]
    │   │   → Outer loop over coins; inner forward sweep: dp[i] += dp[i-c]. This
    │   │     ensures [1,2] and [2,1] are the same combination
    │   └── Perfect Squares [M]
    │       → dp[i] = min(dp[i - k²] + 1) for all k² ≤ i. Base: dp[0]=0
    ├── LCS Family
    │   ├── Longest Common Subsequence [M]
    │   │   → If s1[i-1]==s2[j-1]: dp[i][j] = dp[i-1][j-1]+1; else max(dp[i-1][j],
    │   │     dp[i][j-1]). Space: roll to one row
    │   ├── Edit Distance [M]
    │   │   → If chars match: dp[i-1][j-1]; else 1 + min(replace=dp[i-1][j-1],
    │   │     delete=dp[i-1][j], insert=dp[i][j-1]). Base: dp[i][0]=i, dp[0][j]=j
    │   ├── Longest Palindromic Subsequence [M]
    │   │   → If s[i]==s[j]: dp[i][j] = dp[i+1][j-1]+2; else max(dp[i+1][j],
    │   │     dp[i][j-1]). Fill diagonals outward (length 1→2→…→n)
    │   ├── Minimum ASCII Delete Sum for Two Strings [M]
    │   │   → If s1[i-1]==s2[j-1]: dp[i][j]=dp[i-1][j-1]; else
    │   │     min(dp[i-1][j]+ord(s1[i-1]), dp[i][j-1]+ord(s2[j-1])). Base: prefix
    │   │     ASCII sums
    │   └── Minimum Window Subsequence (LC 727) [M]
    │       → dp[i][j] = dp[i-1][j-1] if s[i]==t[j] else dp[i-1][j]. Base: dp[i][0]
    │         = i when s[i]==t[0]. When dp[i][len(t)-1] is valid, compute window
    │         length = i - dp[i][len(t)-1] + 1 and track minimum
    ├── Grid DP
    │   ├── Unique Paths [M]
    │   │   → dp[j] += dp[j-1] for each row. Base: all 1s initially (single path
    │   │     along edges)
    │   ├── Unique Paths II (With Obstacles) [M]
    │   │   → Same recurrence; set dp[j]=0 when obstacleGrid[i][j]==1. Careful with
    │   │     first row/col initialization
    │   ├── Minimum Path Sum [M]
    │   │   → dp[j] = grid[i][j] + min(dp[j], dp[j-1]). Initialize first row as
    │   │     prefix sums
    │   └── Maximal Square [M]
    │       → If matrix[i][j]=='1': dp[i][j] = min(dp[i-1][j], dp[i][j-1],
    │         dp[i-1][j-1]) + 1. Roll to two rows or 1D
    ├── Interval DP
    │   ├── Burst Balloons [H]
    │   │   → For each k in (i,j): dp[i][j] = max(dp[i][k] +
    │   │     nums[i]*nums[k]*nums[j] + dp[k][j]). Fill by increasing interval length
    │   ├── Palindrome Partitioning II (Minimum Cuts) [M]
    │   │   → dp[i] = min(dp[j-1]+1) for all j ≤ i where s[j..i] is palindrome;
    │   │     dp[j-1]=-1 when j=0
    │   └── Strange Printer [M]
    │       → Base: dp[i][i]=1. For i<j: start with dp[i][j] = dp[i+1][j] + 1
    │         (print s[i] alone, then recurse). If s[i]==s[k] for some k in (i,j]:
    │         dp[i][j] = min(dp[i][j], dp[i+1][k] + dp[k+1][j]) — merge s[i] with
    │         s[k]'s turn
    ├── Tree DP
    │   ├── Diameter of Binary Tree [M]
    │   │   → DFS returns depth; at each node diameter = max(diameter, left_depth +
    │   │     right_depth). Return max(left, right) + 1
    │   ├── Binary Tree Maximum Path Sum [H]
    │   │   → left_gain = max(0, dfs(left)), right_gain = max(0, dfs(right)). ans =
    │   │     max(ans, node.val + left_gain + right_gain). Return node.val +
    │   │     max(left_gain, right_gain)
    │   └── House Robber III [M]
    │       → rob = node.val + left_skip + right_skip; skip = max(left_rob,
    │         left_skip) + max(right_rob, right_skip)
    ├── State Machine DP
    │   ├── Best Time to Buy and Sell Stock (All Variants) [M]
    │   │   → I: track min price; II: sum uphill diffs; III/IV: buy/sell state
    │   │     machine for k txns
    │   ├── Stock with Cooldown [M]
    │   │   → held = max(held, rest - price), sold = held + price, rest = max(rest,
    │   │     sold)
    │   └── Stock with Transaction Fee [M]
    │       → hold = max(hold, cash - price), cash = max(cash, hold + price - fee)
    ├── Bitmask / Digit DP
    │   ├── Shortest Path Visiting All Nodes [M]
    │   │   → Enqueue (1 << i, i) for each node i. BFS level = distance. Terminate
    │   │     when mask == (1<<n)-1
    │   ├── Numbers At Most N Given Digit Set [M]
    │   │   → For k < len(n_str) digits: |digits|^k choices. For k == len(n_str):
    │   │     iterate digit by digit, count choices where current digit < bound
```

```
    digit, then check tight equality
└── Super Egg Drop [H]
    → WHAT (inverted):: dp[m][k] = dp[m-1][k-1] + dp[m-1][k] + 1. Find
      minimum m where dp[m][k] >= n
├── Longest Increasing Subsequence (LIS) Family
    ├── Longest Increasing Subsequence [M]
    → For each x in nums, binary search tails for the first element >= x.
      If found, replace it with x; otherwise append x. Answer = len(tails)
    ├── Number of LIS [M]
    → For each i, scan j < i. If nums[j] < nums[i]: if length[j]+1 >
      length[i], update both; if equal, add count[j] to count[i]. Answer =
      sum of count[i] where length[i] == max_length
    └── Longest Bitonic Subsequence [M]
    → Compute lis left-to-right O(n²), lds right-to-left O(n²). Answer =
      max(lis[i] + lds[i] - 1)
├── String DP
    ├── Distinct Subsequences [M]
    → If s[i-1] == t[j-1]: dp[i][j] = dp[i-1][j-1] + dp[i-1][j] (use or
      skip). Else: dp[i][j] = dp[i-1][j]. Base: dp[i][0] = 1 for all i
    ├── Interleaving String [M]
    → dp[i][j] = (dp[i-1][j] and s1[i-1]==s3[i+j-1]) or (dp[i][j-1] and
      s2[j-1]==s3[i+j-1]). Base: dp[0][0] = True
    └── Regular Expression Matching [H]
    → If p[j-1] == '*': dp[i][j] = dp[i][j-2] (zero uses) or (dp[i-1][j]
      and (p[j-2]=='.' or p[j-2]==s[i-1])) (one+ uses). Else: dp[i][j] =
      dp[i-1][j-1] and (p[j-1]=='.' or p[j-1]==s[i-1])
├── Probability / Expected Value DP
    ├── Knight Probability in Chessboard [M]
    → Start with probability 1 at (r, c). Each step: new dp[ni][nj] +=
      dp[i][j] / 8 for each valid knight move. Repeat k times; answer =
      sum(dp)
    └── New 21 Game [M]
    → Base dp[0] = 1. Maintain window_sum. For x in 1..n: dp[x] =
      window_sum / maxPts. If x < k, add dp[x] to window; if x >= maxPts,
      subtract dp[x - maxPts]. Answer = sum(dp[k..n])
├── Bitmask DP
    ├── Shortest Path Visiting All Nodes (TSP Bitmask DP) [M]
    → BFS (unweighted): enqueue all (node, 1<<node) with distance 0. Expand
      neighbours; stop when mask == (1<<n)-1. For weighted TSP: dp[mask |
      (1<<u)][u] = min(dp[mask][v] + w(v,u)) over all v in mask
    └── Partition to K Equal Subset Sums [M]
    → target = total / k. Iterate all masks in order. For each set mask,
      compute current_sum = sum of selected elements % target. Try adding
      each unselected element; if it fits, dp[mask | (1<<i)] = True. Answer =
      dp[(1<<n)-1]
├── Digit DP
    └── Count Numbers with Unique Digits [M]
    → dp[0] = 1. For length i: first digit has 9 choices (1–9); each
      subsequent digit has 10 - (i-1) choices (avoid used). dp[i] = 9 * 9 * 8
      * ... * (10-i+1). Accumulate sum
├── Game Theory DP
    ├── Stone Game [M]
    → dp[i][j] = max(piles[i] - dp[i+1][j], piles[j] - dp[i][j-1]). Base:
      dp[i][i] = piles[i]. Alice wins iff dp[0][n-1] > 0
    ├── Stone Game II [M]
    → Precompute suffix sums. dp[i][m] = suffix[i] - min(dp[i+x][max(m,x)]
      for x in 1..2m) — the current player takes whatever minimises the
      opponent's haul. Fill right-to-left
    └── Predict the Winner [M]
    → Fill by increasing subarray length. Player 1 wins iff dp[0][n-1] >= 0
├── DP on Sequences
    ├── Jump Game II [M]
    → Iterate; update farthest = max(farthest, i + nums[i]). When i ==
      current_end and not at last index: increment jumps, set current_end =
      farthest
    ├── Maximum Product Subarray [M]
    → At each element x: max_prod, min_prod = max(x, max_prod*x,
      min_prod*x), min(x, max_prod*x, min_prod*x). Update global answer with
      max_prod
    └── Arithmetic Slices II — Subsequence [M]
    → weak: For each pair (j, i) with j < i, d = nums[i] - nums[j]:
      dp[i][d] += dp[j].get(d, 0) + 1. The +1 starts a new weak subsequence
      (j,i). Each existing weak subseq extended to length ≥ 3 contributes
      dp[j][d] to the answer
└── Dynamic Programming — Hard Problems
    ├── Matrix Chain Multiplication [M]
    → Fill by increasing interval length (length 1 has cost 0). Outer loop:
      length from 2 to n. Inner loops: i, then k
    ├── Super Egg Drop (LC 887) [M]
    → Increment m until dp2[m][k] >= n. Since m <= n and k <= n, the loops
      terminate
    └── Russian Doll Envelopes (LC 354) [M]
    → bisect_left on the tails array — same as LC 300 LIS
```

graph-algorithms.md

```
coding/algorithms/graph-algorithms.md
└── graph-algorithms.md
    ├── Dijkstra's Algorithm
    │   ├── Network Delay Time [M]
    │   │   → Min-heap of (dist, node). Lazy deletion guard if d > dist[u]:
    │   │     continue discards stale heap entries. Answer = max(dist.values()); if
    │   │     any node has inf distance, return -1
    │   ├── Swim in Rising Water [M]
    │   │   → Heap entry (max_elevation_on_path, r, c). At each step, cost(u→v) =
    │   │     max(current_max, grid[v]). The first time we pop (N-1,N-1) is the
    │   │     answer
    │   ├── Path with Maximum Probability [M]
    │   │   → Negate heap values for max-heap (or use -prob). Relaxation: prob[u] *
    │   │     w > prob[v] → update. All probabilities in [0,1] — no negative-weight
    │   │     issues
    │   ├── Evaluate Division (LC 399) [M]
    │   │   → Build adjacency list {node: [(neighbor, weight)]}. BFS with (node,
    │   │     product) in queue. Track visited. Return product when dst found
    │   └── Cheapest Flights Within K Stops (Dijkstra variant) [M]
    │       → Heap (cost, node, stops). Mark visited as (node, stops) pair.
    │         Bellman-Ford is simpler for this problem; Dijkstra works but requires
    │         careful pruning
    ├── Bellman-Ford
    │   ├── Cheapest Flights Within K Stops [M]
    │   │   → Critical — use a copy of prices each round (temp = prices[:]).
    │   │     Without the copy, a single round might chain multiple hops, violating
    │   │     the hop bound
    │   └── Find the City with the Smallest Number of Neighbo... [M]
    │       → Initialize diagonal to 0, direct edges to weight, rest to inf. k must
    │         be the outermost loop. After running, count neighbors within threshold
    │         for each city
    ├── Topological Sort (BFS — Kahn's)
    │   ├── Course Schedule [M]
    │   │   → Build adjacency list and in-degree array. BFS from zero-in-degree
    │   │     nodes; decrement neighbors. If processed == numCourses → no cycle
    │   ├── Course Schedule II [M]
    │   │   → Collect dequeued nodes into order. If len(order) == numCourses →
    │   │     valid. Otherwise cycle exists
    │   ├── Alien Dictionary [H]
    │   │   → For each pair (words[i], words[i+1]), find first differing character
    │   │     — adds directed edge. Invalid: if word A is a proper prefix of word B
    │   │     but A appears after B. Cycle in graph → ""
    │   ├── Sequence Reconstruction (Check Unique Topo Order) [M]
    │   │   → If at any point the queue has more than one element → multiple valid
    │   │     orderings → not unique. Also verify the final order equals nums
    │   └── Find All Possible Recipes from Given Supplies [M]
    │       → Treat recipes and supplies as nodes. Edges: ingredient → recipe
    │         (ingredient must precede recipe). Initialize queue with all supply
    │         nodes
    ├── Strongly Connected Components / Bridges
    │   ├── Critical Connections in a Network (Tarjan's Bridges) [M]
    │   │   → DFS from any node. When backtracking from child v to parent u: low[u]
    │   │     = min(low[u], low[v]). For already-visited back edges (non-parent):
    │   │     low[u] = min(low[u], disc[v]). Bridge condition: low[v] > disc[u]
    │   ├── Strongly Connected Components (Kosaraju's Algorithm) [M]
    │   │   → Pass 1 — DFS original graph, push nodes to stack in finish order.
    │   │     Pass 2 — reverse all edges, pop from stack, DFS the reversed graph;
    │   │     each connected component found = one SCC
    │   └── Find Eventual Safe States (Reverse Graph / Kahn's) [M]
    │       → outdegree[u] = original out-degree. Initialize queue with
    │         outdegree[u] == 0. When a node is processed safe, decrement
    │         predecessors' outdegree; add them if 0
    ├── Cycle Detection in Directed Graph
    │   └── Detect Cycle in Directed Graph (DFS 3-Color) [M]
    │       → For each unvisited node, run DFS. Mark GRAY on entry, BLACK on exit.
    │         If we ever encounter a GRAY neighbor, we've found a cycle
    ├── Eulerian Path
    │   └── Reconstruct Itinerary (Hierholzer's) [M]
    │       → Sort each adjacency list in reverse order so .pop() gives the
    │         lexicographically smallest destination. DFS; when stuck (no outgoing
    │         edges), append to result. Reverse result at the end
    ├── 0-1 BFS
    │   ├── Minimum Cost to Make at Least One Valid Path in a... [M]
    │   │   → Each cell has one free neighbor (the direction it points). All other
    │   │     3 neighbors cost 1. Standard Dijkstra also works but 0-1 BFS is
    │   │     asymptotically better
    │   └── Open the Lock (Unweighted BFS Variant) [M]
    │       → Encode combinations as strings. Skip deadends. Mark visited by adding
    │         to a set. Start with "0000" — if it's a deadend, return -1 immediately
    ├── Multi-source BFS / Special BFS
    │   ├── Word Ladder (BFS on Implicit Graph) [H]
    │   │   → Remove visited words from word_set immediately (not just a visited
    │   │     set) to prevent revisits efficiently
    │   ├── Word Ladder II (All Shortest Transformation Seque... [H]
    │   │   → BFS level-by-level. For each word, generate all 1-letter variants in
    │   │     the word set. Record parents[new_word].add(word). Remove words from the
    │   │     set only after the full level is processed (so multiple parents at the
    │   │     same level can be recorded). Then DFS from endWord back to beginWord
    │   │     using the parents map
    │   └── Shortest Path in Binary Matrix [M]
    │       → Check both endpoints are 0 before starting. Distance = BFS level when
    │         target is first reached
    ├── Bipartite
    │   ├── Is Graph Bipartite? [M]
    │   │   → Must handle disconnected components — start BFS from every unvisited
    │   │     node
    │   └── Possible Bipartition [M]
    │       → Identical to isBipartite — just build the graph first from dislikes
    ├── Minimum Spanning Tree — Prim's Algorithm
    │   └── Min Cost to Connect All Points (LC 1584) [M]
    │       → Start from node 0. Min-heap stores (cost, node). Pop cheapest; if
    │         already visited, skip. Add its cost to total. Push all unvisited
    │         neighbors with Manhattan distance as cost. Repeat until all n nodes
    │         visited
    ├── Graph Coloring
    │   └── M-Coloring Problem (Backtracking) [M]
    │       → Try each color for the current node. Before assigning, check all
    │         neighbors — if a neighbor already has that color, skip. If all m colors
    │         fail → backtrack. If all nodes assigned → return True
    └── Graph Algorithms — More Problems
        ├── Shortest Path Visiting All Nodes (LC 847) [M]
        │   → Visited set: {(node, mask)}. Dequeue state, try all neighbours.
        │     Update mask with mask | (1 << neighbour)
        ├── Word Ladder II (LC 126) [M]
```

```
    → Remove words from word_set only after the full layer is processed —
      prevents cutting off valid same-layer paths
└── Travelling Salesman Problem — Bitmask DP [M]
    → Start with dp[1][0] = 0 (started at city 0). Answer:
      min(dp[full_mask][i] + dist[i][0]) for all i
```

greedy.md

```
coding/algorithms/greedy.md
└── greedy.md
    ├── Interval Greedy
    │   ├── Merge Intervals [M]
    │   │   → merged[-1][1] = max(merged[-1][1], end) when start ≤ merged[-1][1]
    │   ├── Non-overlapping Intervals (Minimum number to remove) [M]
    │   │   → Track last_end; if start >= last_end, keep (update last_end = end);
    │   │     else remove (increment count)
    │   ├── Meeting Rooms II [M]
    │   │   → Heap size at the end = rooms needed
    │   ├── Minimum Number of Arrows to Burst Balloons [H]
    │   │   → Arrow at end; skip all balloons with start ≤ end; next arrow at the
    │   │     next un-burst balloon's end
    │   ├── Video Stitching [M]
    │   │   → Two pointers: cur_end (coverage end), farthest (best extension seen).
    │   │     When current clip starts > cur_end, coverage has a gap — return -1
    │   ├── Minimum Taps to Open to Water a Garden [M]
    │   │   → Pre-process: for each position i, compute interval [max(0, i-r),
    │   │     min(n, i+r)]; sort by start; run jump-game-style greedy
    │   └── Minimum Number of Groups for Non-Overlapping Inte... [M]
    │       → Identical to Meeting Rooms II solution
    ├── Simple Greedy
    │   ├── Assign Cookies (LC 455) [M]
    │   │   → Sort g and s. Two pointers i (children), j (cookies). If s[j] >=
    │   │     g[i]: i++, j++. Else j++. Return i
    │   └── Largest Perimeter Triangle (LC 976) [M]
    │       → Sort desc. For i in range(len-2): if nums[i] < nums[i+1]+nums[i+2]:
    │         return sum of these three. Return 0
    ├── Jump / Coverage Greedy
    │   ├── Jump Game (can reach?) [M]
    │   │   → Single pass; early exit the moment current index exceeds max_reach
    │   ├── Jump Game II (minimum jumps) [M]
    │   │   → Loop only to n-2 (last index doesn't need a jump from it)
    │   ├── Jump Game III [M]
    │   │   → visited set prevents cycles. Return False if queue exhausts without
    │   │     finding 0
    │   ├── Jump Game VI (DP + Deque) [M]
    │   │   → Deque stores indices in decreasing dp-value order; pop front when out
    │   │     of window, pop back when dp[i-1] ≥ dp[deque.back()]
    │   └── Minimum Refueling Stops (LC 871) [M]
    │       → Push all reachable stations' fuels into max-heap as you pass them.
    │         When fuel < 0: if heap empty return -1. Pop max fuel, add to tank,
    │         increment stops. Continue until reach target
    ├── Scheduling
    │   ├── Task Scheduler [M]
    │   │   → Pure math; no simulation needed
    │   ├── Candy (LC 135) [M]
    │   │   → 1. Init candies = [1]*n. 2. Left pass: if ratings[i] > ratings[i-1]:
    │   │     candies[i] = candies[i-1]+1. 3. Right pass: if ratings[i] >
    │   │     ratings[i+1]: candies[i] = max(candies[i], candies[i+1]+1). 4. Return
    │   │     sum(candies)
    │   ├── Reorganize String [M]
    │   │   → Track prev (last placed char) — if top of heap == prev, temporarily
    │   │     swap with second-most-frequent
    │   └── Rearrange String k Distance Apart (Rearrange Barc... [M]
    │       → Use a queue to enforce cooldown: after using a character, re-push
```

```
        only after k steps
├── String / Array Greedy
│   ├── Gas Station [M]
│   │   → Single pass. Feasibility check built into the same pass via total sum
│   ├── Trapping Rain Water (greedy view) [H]
│   │   → Advance the pointer with the smaller current boundary; maintain
│   │     running max for each side
│   ├── GCD of Strings [M]
│   │   → math.gcd(m, n)
│   ├── Connect Sticks (Huffman / Min Cost) [M]
│   │   → n−1 merges; each O(log n); total O(n log n)
│   ├── Minimum Difference After Operations [M]
│   │   → For 3 moves: 4 splits — (0,3),(1,2),(2,1),(3,0)
│   └── Partition Labels (LC 763) [M]
│       → Build last[c] = last index of character c. Sweep left to right.
│         Maintain current partition end = max(last[c] for c in current
│         partition). When i == end: partition complete, record size, start new
├── Sorting-Based Greedy
│   ├── Queue Reconstruction by Height (LC 406) [M]
│   │   → result.insert(k, person) — O(n) per insert, but n is small enough;
│   │     taller people already placed are unaffected by later insertions
│   ├── IPO (Maximize Capital, LC 502) [M]
│   │   → Sort projects by capital. For each of k steps: push all projects with
│   │     capital[i] ≤ W into max-heap; pop the most profitable; add to W. If
│   │     max-heap empty, can't proceed
│   └── Activity Selection (Maximum Non-Overlapping Inter... [M]
│       → Track last_end; when start >= last_end, keep the interval and update
│         last_end = end
└── Greedy — Interval and Coverage Problems
    ├── Video Stitching (LC 1024) [M]
    │   → Iterate through sorted clips. If clip_start > cur_end, return -1
    │     (gap). Update farthest. When we've exhausted clips for this jump, set
    │     cur_end = farthest, increment count
    └── Minimum Taps to Water a Garden (LC 1326) [M]
        → Build intervals (max(0, i - ranges[i]), min(n, i + ranges[i])) for
          each tap. Sort. Apply the Video Stitching greedy
```

maths.md

```
coding/algorithms/maths.md
└── maths.md
    ├── Number Theory — Primes / Sieve
    │   ├── Count Primes (Sieve of Eratosthenes) [M]
    │   │   → Initialize all True. Set is_prime[0]=is_prime[1]=False. For each p
    │   │     from 2 to √n: if is_prime[p], mark p*p, p*p+p, ... False. Starting at
    │   │     p² (not 2p) because smaller multiples were already marked by smaller
    │   │     primes
    │   └── Closest Prime Numbers in Range [M]
    │       → Standard sieve up to right. Collect primes in range. If fewer than 2,
    │         return [-1, -1]. Linear scan for minimum gap between consecutive primes
    ├── GCD / LCM
    │   ├── GCD of Strings [M]
    │   │   → Check s1 + s2 == s2 + s1. If True, gcd_len = gcd(len(s1), len(s2));
    │   │     return s1[:gcd_len]. If False, return ""
    │   ├── Find Greatest Common Divisor of Array [M]
    │   │   → Single pass to find min and max; apply Euclidean GCD
    │   ├── Simplified Fractions [M]
    │   │   → Double loop; GCD check per pair. O(n²) pairs, each O(log n) GCD check
    │   └── Water Jug Problem (Bézout's Identity) [M]
    │       → Check both conditions. No simulation needed
    ├── Modular Arithmetic
    │   ├── Pow(x, n) (Fast Exponentiation with Mod) [M]
    │   │   → If n negative: x = 1/x, n = -n. While n > 0: if LSB set, multiply
    │   │     result by x; square x; right-shift n
    │   ├── Super Pow (Modular Exponentiation, Non-Prime Mod) [M]
    │   │   → 1337 = 7 × 191 (not prime), so Fermat's little theorem doesn't
    │   │     directly apply — use direct modular exponentiation
    │   ├── Count Good Numbers [M]
    │   │   → even_count = (n + 1) // 2 (positions 0, 2, 4, ...); odd_count = n //
    │   │     2. Fast pow for each
    │   └── Modular Inverse (Extended Euclidean) [M]
    │       → extended_gcd(a, m) → (g, x, y). Inverse = x % m if g == 1, else no
    │         inverse
    ├── Combinatorics / nCr
    │   ├── Pascal's Triangle [M]
    │   │   → Row i has i+1 elements. row[j] = prev[j-1] + prev[j]. Edges are
    │   │     always 1
    │   ├── Pascal's Triangle II [M]
    │   │   → row[j] += row[j-1] from j = len(row)-1 down to 1; append 1 at end
    │   │     each iteration
    │   └── Unique Paths (Combinatorics Approach) [M]
    │       → Python's math.comb computes this exactly in O(min(m,n)) time without
    │         overflow using integer arithmetic
    ├── Geometric / Statistical
    │   ├── Max Points on a Line (GCD-Based Slope) [M]
    │   │   → For each pair (anchor, other): dx = other.x - anchor.x, dy = other.y
    │   │     - anchor.y. Normalize: g = gcd(|dy|, |dx|), slope = (dy//g, dx//g).
    │   │     Force canonical sign: if dx < 0, negate both. Duplicates (same point)
    │   │     counted separately
    │   └── Minimum Moves to Equal Array Elements (Median) [M]
    │       → Sort nums. Median = nums[n//2]. Answer = sum(|x - median|)
    ├── Digit / Sequence Math
    │   ├── Factorial Trailing Zeroes [M]
    │   │   → Iteratively add n // (5^k) while 5^k ≤ n
    │   ├── Integer Square Root (Newton's Method) [M]
```

```
    │   → lo=0, hi=x. While lo ≤ hi: mid=(lo+hi)//2; if mid*mid ≤ x, try larger
    │     (lo=mid+1); else smaller (hi=mid-1)
    ├── Nth Digit [M]
    │   → For each digit count d: group has 9 × 10^(d-1) numbers contributing d
    │     × 9 × 10^(d-1) digits. When n falls in group d: num = 10^(d-1) +
    │     (n-1)//d; digit index within num = (n-1) % d
    ├── Nth Ugly Number (PQ + 3 Pointers) [M]
    │   → Initialize ugly=[1], p2=p3=p5=0. Repeat n-1 times. Advancing all tied
    │     pointers prevents duplicates
    ├── Sum of Divisors with 4 Divisors [M]
    │   → For each num, trial divide. If exactly 4 divisors, add their sum to
    │     result
    ├── Min Moves to Equal Array Elements II (Median) [M]
    │   → median: Same as "Minimum Moves to Equal Array Elements" variant above
    └── Count Subarrays Divisible by K [M]
        → Initialize remainder_count = {0: 1} (empty prefix). For each element,
          update prefix_sum, compute r = prefix_sum % k; add
          remainder_count.get(r, 0) to answer; increment remainder_count[r].
          Handle negative remainders: r = (prefix_sum % k + k) % k
├── Number Encoding / Conversion
│   ├── Integer to Roman [M]
│   │   → 13 symbols: 1000→M, 900→CM, 500→D, 400→CD, 100→C, 90→XC, 50→L, 40→XL,
│   │     10→X, 9→IX, 5→V, 4→IV, 1→I
│   ├── Roman to Integer [M]
│   │   → For each character (except last): if its value is less than the next
│   │     character's value, subtract; else add. Add the last character
│   │     unconditionally
│   ├── Excel Sheet Column Number [M]
│   │   → result = 0. For each char c: result = result * 26 + (ord(c) -
│   │     ord('A') + 1)
│   ├── Happy Number [M]
│   │   → Floyd's two-pointer on the implicit sequence — slow moves one step,
│   │     fast moves two steps. Cycle iff slow == fast; happy iff that meeting
│   │     point is 1
│   ├── Palindrome Number [M]
│   │   → while x > reversed_half: reversed_half = reversed_half * 10 + x % 10;
│   │     x //= 10. Then x == reversed_half (even) or x == reversed_half // 10
│   │     (odd length)
│   └── Multiply Strings [M]
│       → Iterate i from end of num1, j from end of num2. Final answer: strip
│         leading zeros; "0" if all zeros
├── Probability / Sampling
│   ├── Random Pick with Weight [M]
│   │   → bisect_left on prefix sums after multiplying random by total; or
│   │     bisect_right on the raw prefix sum array after scaling
│   └── Reservoir Sampling [M]
│       → Prove invariant: after seeing i items, each item has probability k/i
│         of being in reservoir. Inductive step: item i+1 chosen with prob
│         k/(i+1); each existing item survives with prob 1 - (k/(i+1)) × (1/k) =
│         i/(i+1); combined probability for old items: k/i × i/(i+1) = k/(i+1). ✓
└── Mathematics — More Problems
    ├── Water Jug Problem (LC 365) [M]
    │   → math.gcd(x, y) — Python standard library
    └── Matrix Exponentiation — Fibonacci in O(log n) [M]
        → Base matrix M = [[1,1],[1,0]]. Multiply using 2×2 matrix multiply.
          Return result[0][1] which is F(n)
```

recursion.md

```
coding/algorithms/recursion.md
└── recursion.md
    ├── Foundation — Include/Exclude
    │   ├── Subsets (Power Set) [M]
    │   │   → At index i, add current path to results, then recurse with i+1 after
    │   │     optionally appending nums[i]. Or equivalently: iterate from i to n,
    │   │     choose nums[j], recurse from j+1
    │   ├── Permutations [M]
    │   │   → Swap nums[i] with nums[start], recurse with start+1, swap back
    │   │     (in-place backtracking preserves O(1) space overhead per level)
    │   └── Binary Tree Paths [M]
    │       → Base case = leaf node → append path to result. Recursive case =
    │         recurse left and right with extended path string
    ├── Divide and Conquer (via Recursion)
    │   ├── Merge Sort [M]
    │   │   → Base case = length ≤ 1. Split, conquer left, conquer right, merge in
    │   │     O(n) with two-pointer technique
    │   └── Fibonacci (Memoized Recursion vs DP) [M]
    │       → Memoize using a dict or @lru_cache. For O(1) space, use two variables
    ├── Pruning & Constraints
    │   ├── Combination Sum (Unbounded) [M]
    │   │   → Recurse with (start=i, remaining-candidates[i]). Backtrack by
    │   │     popping. Sort candidates for early termination
    │   ├── Combination Sum II (No Reuse) [M]
    │   │   → if j > start and candidates[j] == candidates[j-1]: continue — only
    │   │     skip siblings, not the first occurrence at a level
    │   ├── Generate Parentheses [M]
    │   │   → Add ( if open < n; add ) if close < open. Leaf = string of length 2n
    │   ├── Word Search (Grid Backtracking) [M]
    │   │   → Mark board[r][c] with a sentinel (e.g., '#') before recursing,
    │   │     restore after. Check bounds and match before recursing
    │   ├── Palindrome Partitioning [M]
    │   │   → Precompute is_pal in O(n²). DFS: for each end ≥ start, if
    │   │     is_pal[start][end], add substring and recurse from end+1
    │   └── Letter Case Permutation [M]
    │       → At index i, if digit: recurse with i+1. If letter: set lowercase,
    │         recurse, set uppercase, recurse
    ├── Constraint Satisfaction
    │   ├── N-Queens [M]
    │   │   → Track sets cols, diag1 (r-c), diag2 (r+c). For each row, try each
    │   │     column not in any set; add to sets, recurse, remove
    │   ├── Sudoku Solver [H]
    │   │   → Precompute sets for each row, column, and 3×3 box. At each empty
    │   │     cell, try only valid digits; backtrack immediately on failure
    │   ├── Word Search II (Trie + Backtracking) [H]
    │   │   → At each cell, check trie_node.children[char]. If present, descend. If
    │   │     trie_node.word, add to results. Mark visited, recurse 4 directions,
    │   │     unmark. Prune exhausted Trie subtrees
    │   ├── Unique Binary Search Trees II [M]
    │   │   → For each root k in [lo, hi], generate all left subtrees (using
    │   │     lo..k-1) and all right subtrees (using k+1..hi), combine every pair
    │   ├── Expression Add Operators [M]
    │   │   → For each position, try each multi-digit number (avoid leading zeros).
    │   │     On *: curr_val = curr_val - prev_operand + prev_operand * num; on +/-:
    │   │     standard addition
    │   └── Robot Room Cleaner [M]
```

```
│       → Try each of 4 directions (relative to current heading using direction
│         vectors). On return: turn 180°, move forward, turn 180° to restore
│         position+heading
├── Foundation — Recursion Fundamentals
│   ├── Binary Search (Recursive) [M]
│   │   → Pass lo, hi as parameters. Base case: lo > hi → return -1. No extra
│   │     space beyond call stack
│   ├── Tower of Hanoi [M]
│   │   → hanoi(n-1, src, aux, dst) → move disk n → hanoi(n-1, aux, src, dst)
│   ├── Print All Subsequences [M]
│   │   → Carry a running path. At index == n, print path. Recurse both
│   │     branches at each step
│   └── Josephus Problem [M]
│       → Memoize or convert to iteration to avoid O(n) stack depth
├── Graph — Recursive Traversal
│   └── All Paths from Source to Target [M]
│       → Backtrack by appending node, recursing through neighbors, then
│         popping
├── String Recursion
│   ├── Decode String (Recursive) [M]
│   │   → Pass a mutable index (via list). Accumulate digits for k, accumulate
│   │     chars for the current segment, recurse on [, multiply result on ]
│   ├── Flatten Nested List Iterator [M]
│   │   → Single recursive function that iterates over the current list and
│   │     dispatches based on element type
│   └── Nested List Weight Sum [M]
│       → Start with depth=1. At each integer, accumulate val * depth. At each
│         list, recurse with depth + 1
├── Dynamic Programming Foundations (Recursive + Memo)
│   ├── Climbing Stairs (Memoized Recursion) [E]
│   │   → @lru_cache or manual dict. Can extend to k steps: ways(n) =
│   │     sum(ways(n-i) for i in 1..k if n-i >= 0)
│   └── Count Good Numbers [M]
│       → Even positions = ceil(n/2) = (n+1)//2. Odd positions = floor(n/2) =
│         n//2. Use recursive fast-power
├── Mutual / Cross-Recursion
│   ├── Wildcard Matching (LC 44) [M]
│   │   → Base cases: both exhausted → True; pattern exhausted → False; string
│   │     exhausted → remaining pattern must be all *. Recursive: if p[j] == '*',
│   │     try skip-star (dp(i, j+1)) or consume-one (dp(i+1, j)). Else if p[j] ==
│   │     '?' or p[j] == s[i], advance both
│   └── Regular Expression Matching (LC 10) [M]
│       → If j+1 < len(p) and p[j+1] == '*': either skip the x* pair (dp(i,
│         j+2)) or, if current chars match, consume one from s (dp(i+1, j)).
│         Otherwise if chars match (. or exact), advance both
├── Structural Tree Recursion
│   ├── Flatten Binary Tree to Linked List (LC 114) [M]
│   │   → Recursively flatten root.left and root.right. Find the rightmost node
│   │     of the flattened left subtree. Attach root.right to it, move the left
│   │     chain to root.right, set root.left = None
│   ├── Construct Binary Tree from Preorder and Inorder T... [M]
│   │   → Root = preorder[0]. Find mid = inorder.index(root.val). Left subtree
│   │     uses preorder[1:mid+1] and inorder[:mid]. Right uses the remainder
│   ├── Serialize and Deserialize Binary Tree (LC 297) [M]
│   │   → Serialize: root.val, serialize(left), serialize(right) joined by a
│   │     delimiter. Deserialize: pop from deque; if "#" return None; else create
│   │     node and recurse for left then right
│   └── Count Complete Tree Nodes (LC 222) [M]
│       → left_h = right_h → left is full, so (1 << left_h) +
│         count(root.right). Otherwise (1 << right_h) + count(root.left)
├── Combinatorial Generation
│   ├── Permutations II (LC 47) [M]
│   │   → Sort nums. At each level, skip nums[i] if nums[i] == nums[i-1] and
│   │     not used[i-1] (sibling was already explored — ensures we always use the
│   │     left duplicate before the right at any level)
│   ├── Subsets II (LC 90) [M]
│   │   → Sort nums. In the loop for i in range(start, n): if i > start and
│   │     nums[i] == nums[i-1], skip. Append path snapshot, continue
│   └── Combinations (LC 77) [M]
│       → Prune early: if remaining elements n - i + 1 < k - len(path), no
│         valid completion possible — skip
├── Divide and Conquer — Advanced
│   ├── Maximum Subarray — D&C (O(n log n)) [E]
│   │   → max_cross = max suffix of left + max prefix of right. Return
│   │     max(max_left, max_right, max_cross)
│   ├── Pow(x, n) — Fast Exponentiation [M]
│   │   → If n == 0 return 1. If n < 0, compute 1 / pow(x, -n). If n is even,
│   │     half = pow(x, n//2); return half * half. If odd, return x * pow(x, n-1)
│   └── Count of Inversions (Merge-Sort Based) [M]
│       → In the merge step, when right[j] < left[i], add len(left) - i to the
│         count (all remaining left elements are greater than right[j])
├── Recursion on Graphs
│   ├── Clone Graph (LC 133) [M]
│   │   → If node already in visited, return its clone. Otherwise create a new
│   │     node, record it in visited, then recursively clone each neighbor and
│   │     append to the new node's neighbors list
│   └── Number of Islands (LC 200) [M]
│       → DFS marks grid[r][c] = '0' then recurses into all 4 directions if
│         in-bounds and == '1'
├── Recursive Parsing / Evaluation
│   ├── Basic Calculator II (LC 227) [M]
│   │   → Use an index pointer (wrapped in a list for mutability) advanced
│   │     through the string. parse_expr calls parse_term repeatedly; parse_term
│   │     calls parse_factor (a number)
│   └── Evaluate Division (LC 399) [M]
│       → DFS from src to dst, multiplying edge weights along the path, using a
│         visited set to avoid cycles. Return accumulated product or -1.0 if
│         destination unreachable
└── Recursion — More Problems
    ├── Predict the Winner (LC 486) [M]
    │   → Memoize with @lru_cache. Return dp(0, n-1) >= 0
    └── Flatten Nested List Iterator (LC 341) [M]
        → Use a stack of (nested_list, index) pairs. _advance() is called by
          hasNext() to ensure the top is an integer
```

sliding-window.md

```
coding/algorithms/sliding-window.md
└── sliding-window.md
    ├── Fixed-Size Window
    │   ├── Find All Anagrams in a String (LC 438) [M]
    │   │   → On adding s[right]: if freq reaches exactly need[c] → matches += 1.
    │   │     On removing s[left]: if freq drops below need[c] → matches -= 1. When
    │   │     matches == len(need) → anagram found
    │   ├── Maximum Average Subarray I (LC 643) [M]
    │   │   → Trivial incremental sum — no auxiliary data structure needed
    │   └── Permutation in String (LC 567) [M]
    │       → When matches == required at any valid window position → return True
    ├── Variable Window — At Most K
    │   ├── Longest Substring Without Repeating Characters (L... [M]
    │   │   → Only jump left if last_seen[c] >= left (character might be outside
    │   │     current window — stale)
    │   ├── Longest Repeating Character Replacement (LC 424) [M]
    │   │   → Key insight: max_count never needs to decrease (we only care about
    │   │     the *best* window seen so far). When we shrink, max_count stays the
    │   │     same, and the window stays the same size or shrinks — we're looking for
    │   │     a *longer* window
    │   ├── Fruits Into Baskets (At Most 2 Distinct) (LC 904) [M]
    │   │   → Window length right - left + 1 after shrinking is the candidate
    │   │     answer
    │   └── Longest Substring with At Most K Distinct Charact... [M]
    │       → Remove key from freq map when its count reaches 0 to keep len(freq)
    │         accurate
    ├── Variable Window — Exactly K
    │   ├── Subarrays with K Different Integers (LC 992) [M]
    │   │   → count(exactly k) = at_most(k) - at_most(k-1)
    │   └── Count Number of Nice Subarrays (LC 1248) [M]
    │       → Window is valid when count of odds ≤ k. Shrink when count > k
    ├── Variable Window — Minimum Length
    │   ├── Minimum Size Subarray Sum (LC 209) [M]
    │   │   → Inner while loop shrinks and records — the answer is updated at the
    │   │     tightest valid window for each right
    │   └── Minimum Window Substring (LC 76) [H]
    │       → need can go negative (excess characters) — only decrement missing
    │         when need[c] was positive (character was still required)
    ├── Sliding Window + Monotonic Deque
    │   ├── Sliding Window Maximum (LC 239) [H]
    │   │   → Before adding i: (1) pop front if it's outside window [i-k+1, i]; (2)
    │   │     pop back while nums[deque[-1]] <= nums[i] (smaller elements can never
    │   │     be max while i is in window). Append i. Record nums[deque[0]] once i >=
    │   │     k-1
    │   └── Sliding Window Minimum (LC variant) [M]
    │       → Only change from max version: reverse comparison nums[dq[-1]] > val
    │         (use >= to maintain strictly increasing, or > for non-strictly)
    ├── Fixed Window — Threshold & Uniqueness
    │   ├── Number of Sub-arrays of Size K and Average ≥ Thre... [M]
    │   │   → Seed with sum of first k elements. Slide: add arr[i], subtract
    │   │     arr[i-k], check threshold
    │   ├── Maximum Sum of Almost Unique Subarray (LC 2841) [M]
    │   │   → Slide in O(1): add right element to freq/sum, remove left element
    │   │     from freq/sum (delete key at 0). Check qualification after each full
    │   │     window
    │   └── Sliding Window Average from Data Stream (LC 346) [M]
    │       → deque.popleft() evicts oldest; deque.append(val) adds newest. No need
    │         to resum — O(1) update
    ├── Variable Window — Max Length (Flip / Delete)
    │   ├── Max Consecutive Ones III (LC 1004) [M]
    │   │   → Answer is right - left + 1 after each valid step — window never
    │   │     shrinks below the best size seen (LC 424 trick not needed here since we
    │   │     do want exact max)
    │   ├── Longest Subarray of 1s After Deleting One Element... [M]
    │   │   → Exact same code as LC 1004 with k=1, subtract 1 from result. Edge: if
    │   │     whole array is ones, deleting one element gives n-1
    │   ├── Minimum Operations to Reduce X to Zero (LC 1658) [M]
    │   │   → Use a variable window (shrink when sum exceeds target, track max
    │   │     length when sum == target). Requires all non-negative integers for
    │   │     monotone shrink property — guaranteed by constraints
    │   └── Binary Subarrays with Sum (LC 930) [M]
    │       → Shrink while sum > k. Handle k < 0 edge case (return 0) to avoid
    │         infinite loop when goal=0
    ├── Variable Window — Minimum Length (All Characters / Distinct)
    │   ├── Minimum Window with All Characters Including Dupl... [M]
    │   │   → On add: decrement need[c]; if it was positive, decrement missing. On
    │   │     remove: increment need[s[left]]; if it becomes positive, increment
    │   │     missing. This correctly handles excess copies
    │   └── Smallest Subarray with Distinct Element Count K [M]
    │       → Shrink: remove nums[left] from freq (delete at 0); stop when
    │         len(freq) < k. Record window size before overshoot
    ├── Sliding Window + Monotonic Deque — Variable Window
    │   ├── Longest Continuous Subarray with Absolute Diff ≤ ... [M]
    │   │   → Shrink by advancing left; pop deque fronts when they equal left (no
    │   │     longer in window). Record right - left + 1 after each valid state
    │   └── Jump Game VI (LC 1696) [M]
    │       → Before computing dp[i]: evict stale front (dq[0] < i - k). After
    │         computing dp[i]: evict back while dp[dq[-1]] <= dp[i]; append i.
    │         Space-optimise by storing dp in original array
    └── More Variable Window Problems
        ├── Subarray Product Less Than K (LC 713) [M]
        │   → Shrink by dividing out nums[left] and advancing left. Handle edge k
        │     <= 1 upfront (product of positives is always ≥ 1)
        ├── Longest Subarray with Sum ≤ K (Nonnegative) [M]
        │   → The while-loop shrink guarantees the window is valid at every right
        │     before recording length
        ├── Minimum Number of Flips to Make Binary String Alt... [M]
        │   → Use a sliding window on s + s. Maintain the count of mismatches with
        │     pattern "010101..." and "101010...". Slide: subtract the outgoing
        │     character's mismatch contribution, add the incoming character's. Track
        │     the minimum
        ├── Grumpy Bookstore Owner (LC 1052) [M]
        │   → Compute base. Slide window: at each step, add customers[right] *
        │     grumpy[right] and subtract customers[right - minutes] * grumpy[right -
        │     minutes]. Track max window extra
        ├── Diet Plan Performance (LC 1176) [M]
        │   → Seed with first k elements. Slide: add right, remove left-k, evaluate
        └── Count Vowel Substrings of a Word (LC 2062) [M]
            → On encountering a consonant, reset left = right + 1 and clear freq.
              exactly(5) = at_most(5) - at_most(4)
```

sorting.md

```
coding/algorithms/sorting.md
└── sorting.md
    ├── Merge Sort Variants
    │   ├── Merge Intervals [M]
    │   │   → For each interval after sort, either extend merged[-1][1] =
    │   │     max(merged[-1][1], end) or append a new interval
    │   ├── Sort List [M]
    │   │   → get_mid severs the list at the middle. Merge uses dummy head and two
    │   │     pointers
    │   ├── Count of Smaller Numbers After Self [M]
    │   │   → Sort (original_index, value) pairs; accumulate into
    │   │     result[original_index] during merge
    │   ├── Count of Range Sum [M]
    │   │   → For each right-half prefix r, maintain two pointers lo, hi on sorted
    │   │     left half: lo = first index where r - left[lo] ≤ upper, hi = first
    │   │     where r - left[hi] < lower. Count = hi - lo
    │   ├── Reverse Pairs [M]
    │   │   → Inner loop: for each left[i], advance j while left[i] > 2 * right[j].
    │   │     Count += j per left element
    │   └── Count Inversions (Merge Sort) [M]
    │       → Recursive merge sort returning (sorted_array, inversion_count).
    │         Standard merge with count accumulation
    ├── QuickSort / QuickSelect
    │   ├── Kth Largest Element in an Array [M]
    │   │   → Target rank = k-1 (0-indexed from largest). Stop when pivot index ==
    │   │     target
    │   ├── K Closest Points to Origin [M]
    │   │   → Partition by dist² ascending; recurse until pivot == k
    │   └── Top K Frequent Elements [M]
    │       → len(nums)+1 buckets (freq ranges 1..n); linear scan backward
    ├── Counting / Radix Sort
    │   ├── Sort Colors [M]
    │   │   → nums[mid]==0: swap(lo,mid), lo++, mid++. nums[mid]==1: mid++.
    │   │     nums[mid]==2: swap(mid,hi), hi-- (don't advance mid — swapped value is
    │   │     unexplored)
    │   └── Maximum Gap (Bucket Sort) [M]
    │       → bucket_idx = (num - min_v) // bucket_width; scan pairs of consecutive
    │         non-empty buckets
    ├── Bucket Sort
    │   └── Sort Characters by Frequency [M]
    │       → At most n distinct frequencies; linear scan of buckets from top
    ├── Custom Comparators
    │   ├── Pancake Sorting [M]
    │   │   → At most 2(n-1) flips total
    │   ├── Queue Reconstruction by Height [M]
    │   │   → Python list.insert(k, person) shifts subsequent elements right —
    │   │     their k values remain valid since all have height ≤ current
    │   ├── Custom Sort String [M]
    │   │   → O(n log n) sort; unranked characters get a default rank beyond the
    │   │     end
    │   ├── Largest Number (custom comparator) [M]
    │   │   → Edge case: all zeros → return "0"
    │   ├── Russian Doll Envelopes (LC 354) [M]
    │   │   → Sort envelopes by (w asc, h desc). Extract heights. Find LIS length
    │   │     using binary search on tails array
    │   └── Wiggle Sort II (LC 324) [M]
    │       → Find median (QuickSelect O(n)). Use 3-way partition (Dutch flag).
    │         Place using index map i → (1+2*i)%(n|1) to interleave
    ├── Interview Classics
    │   ├── H-Index (LC 274) [M]
    │   │   → For each i, if citations[i] >= i+1, update h = i+1. Return max h
    │   │     found
    │   └── Meeting Rooms II (LC 253) [M]
    │       → Sort intervals by start. For each interval: if heap and heap[0] <=
    │         start, heappop (reuse). heappush(end). Return heap size
    ├── Classic Merge Variants
    │   ├── Merge Two Sorted Lists [M]
    │   │   → Dummy head avoids null-checking the result head. After loop, cur.next
    │   │     = l1 or l2
    │   ├── Merge K Sorted Lists [H]
    │   │   → Heap entries are (val, tie_break_id, node) — tie-break prevents
    │   │     comparing ListNode objects on equal values
    │   └── Median of Two Sorted Arrays [H]
    │       → Always binary search on the shorter array. Handle edge cases with
    │         ±inf
    ├── Majority / Frequency
    │   ├── Majority Element (Boyer-Moore) [M]
    │   │   → The surviving candidate after one pass is the majority. (If majority
    │   │     not guaranteed, do a second pass to verify.)
    │   └── Majority Element II [M]
    │       → After the voting pass, verify both candidates have true count > n/3
    │         (the vote may admit false positives for the second candidate)
    ├── Radix / Counting Sort
    │   ├── Sort an Array (Counting Sort Variant) [M]
    │   │   → Shift values by min so indices stay non-negative. Reconstruct the
    │   │     sorted array by iterating the count array
    │   └── Maximum Number After Digit Swaps (LC 2231) [M]
    │       → Collect digits at even indices into a sorted (descending) pool,
    │         refill positions left-to-right from the pool; repeat for odd indices
    ├── Interval / Sweep Line
    │   ├── Insert Interval (LC 57) [M]
    │   │   → Overlap condition: existing.end >= new.start AND existing.start <=
    │   │     new.end
    │   ├── Non-overlapping Intervals (LC 435) [M]
    │   │   → Removals = total - number kept. Ties in end time: keep the one with
    │   │     the earlier end (already handled by sort)
    │   └── Minimum Number of Arrows to Burst Balloons (LC 452) [M]
    │       → A balloon [start, end] is burst by arrow at position pos iff start <=
    │         pos <= end
    ├── Offline Sorting Tricks
    │   ├── Sort Array by Parity (LC 905) [M]
    │   │   → Invariant: everything left of lo is even; everything right of hi is
    │   │     odd
    │   ├── Relative Sort Array (LC 1122) [M]
    │   │   → rank = {v: i for i, v in enumerate(arr2)}. Sort with key = lambda x:
    │   │     rank[x] if x in rank else len(arr2) + x
    │   └── Advantages Shuffle (LC 870) [M]
    │       → Track original indices of nums2 to place answers correctly
    ├── Topological Sort
    │   ├── Course Schedule II (LC 210) [M]
    │   │   → If len(order) < n, a cycle exists → return []
    │   └── Alien Dictionary (LC 269) [M]
    │       → Kahn's BFS topological sort on the character graph. Cycle → return
    │         "". Unconnected characters can appear anywhere
```

```
├── External Sort / K-way Merge
│   ├── Find K Pairs with Smallest Sums (LC 373) [M]
│   │   → At most k pops → O(k log k) after O(min(k, m) log min(k, m)) initial
│   │     heapify
│   └── Kth Smallest Element in a Sorted Matrix (LC 378) [M]
│       → Count function: start at top-right; if matrix[r][c] <= mid → add r+1
│         to count, move right; else move up. O(n) per count call
├── Partial Sort / Order Statistics
│   ├── Kth Largest Element in a Stream (LC 703) [M]
│   │   → Heap size invariant: always ≤ k. After each add, heap[0] = k-th
│   │     largest among all seen elements
│   ├── Find Median from Data Stream (LC 295) [M]
│   │   → On add: push to lo (negate), rebalance by moving top of lo to hi,
│   │     then if len(hi) > len(lo) move top of hi back to lo
│   └── Sliding Window Median (LC 480) [M]
│       → Rebalance: after each add/remove, ensure len(lo) == len(hi) or
│         len(lo) == len(hi) + 1. Lazy removal: pop from heap top while heap[0]
│         is in to_remove
└── Sorting Applications
    └── Maximum Gap (LC 164) [M]
        → Edge cases: if all elements are equal, return 0. If n < 2, return 0
```

string-algorithms.md

```
coding/algorithms/string-algorithms.md
└── string-algorithms.md
    ├── KMP (Knuth-Morris-Pratt)
    │   ├── Implement KMP — Failure Function + Search [M]
    │   │   → lps[i] = length of longest proper prefix of P[0..i] that is also a
    │   │     suffix. On mismatch at pattern position j, jump to lps[j-1]; the text
    │   │     pointer i never moves backward. O(n+m) total because every character is
    │   │     "visited" at most twice
    │   ├── Find All Occurrences of Pattern in Text [M]
    │   │   → The line j = lps[j-1] after a match is the key — it reuses the
    │   │     overlap, so overlapping occurrences like "aaa" in "aaaa" are all found
    │   ├── Repeated Substring Pattern [M]
    │   │   → Build LPS of the full string s. If lps[n-1] > 0 and n % (n -
    │   │     lps[n-1]) == 0, a valid period exists. Equivalently, check if s appears
    │   │     in (s+s)[1:-1]
    │   └── Add Minimum Characters to Make String a Palindrome [M]
    │       → LPS[last] on the concatenated string = length of longest palindromic
    │         prefix. Minimum additions = n - LPS[last]
    ├── Rabin-Karp (Rolling Hash)
    │   ├── Implement Rabin-Karp [M]
    │   │   → Pre-compute BASE^(m-1) mod MOD. On hash match, verify
    │   │     character-by-character to rule out collisions. Double hashing (two
    │   │     independent mod values) reduces false positive probability to
    │   │     ~1/(p₁·p₂)
    │   ├── Longest Duplicate Substring [M]
    │   │   → Binary search [1, n-1]. check(L): slide window of size L, compute
    │   │     hash in O(1) per step, store in set. If duplicate found, record it;
    │   │     search larger L. If not, search smaller. Total: O(n log n)
    │   └── Count Distinct Doubled Substrings [M]
    │       → Maintain two rolling hashes (left window and right window of size L)
    │         simultaneously. When both hashes match, store the canonical string in a
    │         set for deduplication
    ├── Z-Function / Z-Algorithm
    │   ├── Pattern Matching Using Z-Array [M]
    │   │   → Maintain a Z-box [l, r] — the rightmost interval where a match with a
    │   │     prefix is known. For each i: if i <= r, initialize Z[i] = min(r - i +
    │   │     1, Z[i - l]); then try to extend. The total number of character
    │   │     comparisons is O(n+m) because the right boundary r only moves right
    │   └── Repeated String Match [M]
    │       → The minimum repetitions needed is ceil(len(B) / len(A)). If B isn't
    │         found there, try ceil + 1 (B might straddle one more copy). Use KMP or
    │         Z-algorithm for the search to stay O(|A| + |B|)
    ├── Palindrome Algorithms
    │   ├── Longest Palindromic Substring — Expand Around Center [M]
    │   │   → For each i, try both odd-center (i, i) and even-center (i, i+1).
    │   │     Track the longest expansion. No extra space needed
    │   ├── Palindromic Substrings — Count [M]
    │   │   → Same 2n-1 centers. For each center, count the number of valid
    │   │     expansions (each step adds 1 to count)
    │   ├── Palindrome Partitioning II — Minimum Cuts [M]
    │   │   → Pre-compute is_pal[i][j] using expand-around-center in O(n²). Then
    │   │     linear scan for dp[i]. Base: dp[i] = i (cut every character). If
    │   │     s[0..i] is itself a palindrome, dp[i] = 0
    │   └── Manacher's Algorithm — O(n) All Palindromes [M]
    │       → Each character is "extended past" at most once (because r only
    │         increases), so total comparisons = O(n)
```

```
├── Suffix Array / Trie Based
│   ├── Number of Distinct Substrings [M]
│   │   → Distinct substrings = n(n+1)/2 − sum(LCP)
│   └── Longest Common Prefix of All Suffixes [M]
│       → Kasai's key insight: if suffix starting at i has LCP of h with its SA
│         predecessor, then suffix starting at i+1 has LCP ≥ h−1 with its SA
│         predecessor. This means we can start each computation from h−1 and h
│         only ever decreases by 1 between iterations → O(n) total
├── Sliding Window on Strings
│   ├── Longest Substring with K Distinct Characters [M]
│   │   → When freq[s[left]] == 0 after decrement, delete from map — this
│   │     decrements distinct count. Window size at each valid state is a
│   │     candidate answer
│   ├── Permutation in String [M]
│   │   → Track matches = number of characters (out of 26) where window
│   │     frequency equals s1 frequency. Slide window: add right char, remove
│   │     left char, update matches accordingly. Avoids O(26) comparison per step
│   └── Minimum Window Substring [H]
│       → formed increments only when have[c] == need[c] (exact threshold), not
│         on every increment. This makes the condition O(1) to check per step
├── Classic String Problems
│   ├── Valid Palindrome [M]
│   │   → No extra string creation needed — move pointers in-place
│   ├── Valid Anagram [E]
│   │   → One pass to build, one pass to compare. O(n) time. For Unicode inputs
│   │     use Counter (not fixed 26-array)
│   ├── Longest Common Prefix [M]
│   │   → Compare position by position across all strings. Return the prefix up
│   │     to the first mismatch
│   ├── Group Anagrams [M]
│   │   → Sorting each string: O(k log k) per string where k = max length.
│   │     Total: O(nk log k). Alternatively, use a tuple of 26 counts as key:
│   │     O(nk) total
│   ├── Longest Substring Without Repeating Characters [M]
│   │   → Direct index tracking is more efficient than a frequency-decrement
│   │     approach for this problem
│   └── Find All Anagrams in a String [M]
│       → Identical to Permutation in String but collect all starting indices
│         where matches == 26 instead of returning True on first match
├── DP on Strings
│   ├── Regular Expression Matching (LC 10) [M]
│   │   → - Base: dp[0][0] = True. dp[0][j] = True if p[j−1] == '*' and
│   │     dp[0][j−2] == True (star eliminates the preceding element). -
│   │     Transition: if p[j−1] in {s[i−1], '.'}: dp[i][j] = dp[i−1][j−1]. - Else
│   │     if p[j−1] == '*': dp[i][j] = dp[i][j−2] (zero uses) OR (dp[i−1][j] if
│   │     p[j−2] in {s[i−1], '.'}) (one or more uses)
│   ├── Wildcard Matching (LC 44) [M]
│   │   → - Base: dp[0][0] = True. dp[0][j] = True if p[:j] is all '*' (each
│   │     star matches empty). - Transition: if p[j−1] in {s[i−1], '?'}: dp[i][j]
│   │     = dp[i−1][j−1]. - Else if p[j−1] == '*': dp[i][j] = dp[i−1][j] (star
│   │     matches one char) OR dp[i][j−1] (star matches empty)
│   └── Distinct Subsequences (LC 115) [M]
│       → - Base: dp[i][0] = 1 for all i (empty t matched by any prefix of s).
│         dp[0][j] = 0 for j > 0. - Transition: if s[i−1] == t[j−1]: dp[i][j] =
│         dp[i−1][j−1] + dp[i−1][j] (use this char OR skip it). - Else: dp[i][j]
│         = dp[i−1][j] (must skip s[i−1])
└── String Algorithms — More Problems
    ├── Add Minimum Characters to Make a String Palindrome [M]
    │   → Standard O(n²) DP. Can space-optimise to O(n) using two rows
    └── Palindrome Pairs (LC 336) [M]
        → Avoid self-pairing by checking j != i
```

two-pointers.md

```
coding/algorithms/two-pointers.md
└── two-pointers.md
    ├── Opposite Ends (Sorted Array)
    │   ├── Two Sum II — Input Array is Sorted (LC 167) [M]
    │   │   → Each step either finds the answer or eliminates at least one
    │   │     impossible pair. Total: O(n) steps
    │   ├── 3Sum (LC 15) [M]
    │   │   → Skip duplicate anchors (nums[i] == nums[i-1]). On finding a triplet,
    │   │     skip duplicate inner pointers before advancing
    │   ├── 4Sum (LC 18) [M]
    │   │   → Deduplicate both outer anchors separately. Same skip-duplicate
    │   │     pattern as 3Sum on inner pointers
    │   ├── Container With Most Water (LC 11) [M]
    │   │   → if height[lo] <= height[hi]: lo += 1 else hi -= 1. Record max at each
    │   │     step
    │   └── Trapping Rain Water (LC 42) [H]
    │       → if left_max <= right_max: water += left_max - height[lo]; lo += 1
    │         else process right side
    ├── Same Direction (Fast/Slow)
    │   ├── Remove Duplicates from Sorted Array (LC 26) [M]
    │   │   → Each non-duplicate advances write. Duplicates are simply skipped
    │   ├── Remove Element (LC 27) [M]
    │   │   → Alternatively, swap with the end when val is found — useful when val
    │   │     is rare (avoids shifting)
    │   ├── Move Zeroes (LC 283) [M]
    │   │   → Avoids unnecessary writes for zeros — write pointer skips over zero
    │   │     positions and fills at the end
    │   └── Squares of a Sorted Array (LC 977) [M]
    │       → pos = n-1. While lo <= hi: if abs(nums[lo]) >= abs(nums[hi]) →
    │         result[pos] = nums[lo]2; lo += 1; else process hi
    ├── Partition / Dutch National Flag
    │   └── Sort Colors (LC 75) [M]
    │       → Process nums[mid]: if 0 → swap with lo, advance both lo and mid; if 1
    │         → advance mid only; if 2 → swap with hi, decrement hi but do NOT
    │         advance mid (swapped element is unseen)
    ├── Linked List Two Pointers
    │   ├── Linked List Cycle (LC 141) [E]
    │   │   → Check fast and fast.next before each step to avoid null pointer
    │   │     dereference
    │   ├── Find Duplicate Number — Floyd's (LC 287) [M]
    │   │   → Phase 1 uses slow = nums[slow]; fast = nums[nums[fast]]. Phase 2:
    │   │     slow = 0; while slow != fast: slow = nums[slow]; fast = nums[fast]
    │   ├── Middle of the Linked List (LC 876) [M]
    │   │   → Condition while fast and fast.next — for even-length lists, slow
    │   │     stops at the second middle (upper-mid). This matches the problem
    │   │     requirement
    │   └── Remove Nth Node From End of List (LC 19) [M]
    │       → Use a dummy head to handle edge case of deleting the first node.
    │         slow.next = slow.next.next removes the target
    ├── Palindrome Two Pointers
    │   ├── Valid Palindrome II (LC 680) [M]
    │   │   → Helper function checks palindrome in a range. O(n) total — we branch
    │   │     at most once
    │   └── 3Sum Closest (LC 16) [M]
    │       → If s < target → lo += 1 (need larger sum); if s > target → hi -= 1;
    │         if equal → return immediately
    ├── Hash Map Two-Sum Variant
    │   └── 4Sum II (LC 454) [M]
    │       → Two nested loops each O(n²) rather than O(n⁴). This is a hash-map
    │         complement pattern, not classic two-pointer, but pairs with kSum
    │         problems
    ├── Partition Variants
    │   ├── Partition Array According to Given Pivot (LC 2161) [M]
    │   │   → less + equal + greater preserves original relative order within each
    │   │     group
    │   └── Minimum Difference Between Highest and Lowest of ... [M]
    │       → This is a two-pointer fixed window (lo = i, hi = i + k - 1). Single
    │         pass after sort
    ├── Sorted Array / Two-Pointer Greedy
    │   ├── Number of Subsequences That Satisfy the Given Sum... [M]
    │   │   → hi never moves right (as lo increases, the valid range can only
    │   │     shrink), so two-pointer is O(n)
    │   ├── Boats to Save People (LC 881) [M]
    │   │   → Each iteration uses one boat. Count iterations until left > right
    │   ├── Max Number of K-Sum Pairs (LC 1679) [M]
    │   │   → Greedy pairing of smallest + largest exhausts all valid pairs
    │   │     optimally (each element can only be used once)
    │   └── Bag of Tokens (LC 948) [M]
    │       → Always track best = max(best, points) — we may want to stop before
    │         spending all points
    ├── Two-Pointer on Strings
    │   ├── Reverse String (LC 344) [M]
    │   │   → Loop condition lo < hi; each iteration does one swap and two pointer
    │   │     moves
    │   └── Long Pressed Name (LC 925) [M]
    │       → After the loop, i must have consumed all of name
    └── Two Pointers — More Problems
        └── Minimum Operations to Reduce X to Zero (LC 1658) [M]
            → Two-pointer (sliding window): expand right to grow the window, shrink
              left when the window sum exceeds target. Track the maximum window
              length where sum equals target
```

union-find.md

```
coding/algorithms/union-find.md
└── union-find.md
    ├── Basic Union-Find
    │   ├── Number of Connected Components in an Undirected G... [M]
    │   │   → Initialize components = n; decrement by 1 on each successful union
    │   ├── Number of Provinces (Matrix Form) [M]
    │   │   → Only process upper triangle (j > i) to avoid redundant unions and
    │   │     double-decrementing
    │   ├── Graph Valid Tree [M]
    │   │   → Short-circuit if len(edges) != n-1. Process edges; if any union
    │   │     returns False (cycle) → not a tree
    │   ├── Redundant Connection [M]
    │   │   → union returns False when already connected → that's the answer
    │   └── Satisfiability of Equality Equations [M]
    │       → 26 lowercase letters → DSU of size 26. If find(x) == find(y) for a !=
    │         constraint → contradiction
    ├── Weighted / Ranked Union-Find
    │   ├── Accounts Merge (Email Graph) [M]
    │   │   → Map email → index. For each account, union the first email with all
    │   │     subsequent ones. After all unions, group indices by root
    │   ├── Smallest String With Swaps [M]
    │   │   → Group indices by DSU root. For each group, collect and sort
    │   │     characters; assign back to sorted index positions
    │   └── Evaluate Division (Weighted DSU) [M]
    │       → Query C/D: if same root, answer = weight[C] / weight[D]
    ├── MST (Kruskal's)
    │   ├── Min Cost to Connect All Points [M]
    │   │   → Stop early when n-1 edges are added. Prim's with simple array is
    │   │     O(N²) and avoids generating/sorting edges — better for dense graphs
    │   └── Critical Connections / Pseudo-Critical Edges in MST [M]
    │       → Base MST first. For edge e: critical if MST_without_e > base.
    │         Pseudo-critical if MST_with_e_forced == base
    ├── Dynamic / Offline Union-Find
    │   ├── Number of Islands II [M]
    │   │   → Track which cells are land (set). Skip duplicate additions. DSU's
    │   │     union automatically decrements component count on merge
    │   └── Minimize Malware Spread [M]
    │       → Build full DSU. Count infected nodes per component root. The best
    │         removal candidate is the infected node whose component has exactly 1
    │         infected node and the largest size. Tie-break: smallest index
    ├── DSU for Other Problems
    │   ├── Longest Consecutive Sequence (DSU Approach) [M]
    │   │   → Build val → index map. For each value, if val+1 exists, union their
    │   │     indices
    │   └── Path with Maximum Probability (Weighted DSU) [M]
    │       → union(A, B, p): adjust root weight so weight[A] / weight[B] = p.
    │         Valid only when graph is a tree
    ├── Directed Graph Union-Find
    │   └── Redundant Connection II [M]
    │       → Pass 1: record in-degree-2 candidates. Pass 2: DSU union excluding
    │         cand2. If no cycle detected with cand2 excluded → return cand2. If
    │         cycle detected and cand1 exists → return cand1. If cycle and no
    │         candidate → return the cycle edge
    ├── Grid / Coordinate Union-Find
    │   ├── Swim in Rising Water [M]
    │   │   → Create list of (elevation, r, c), sort it. Maintain visited set. For
    │   │     each cell in order, mark visited, union with adjacent visited cells,
    │   │     check connectivity
    │   └── Most Stones Removed with Same Row or Column [M]
    │       → Map rows to [0, 10000] and cols to [10001, 20001]. Union row_r with
    │         col_c for each stone. Count distinct roots among only the stone
    │         positions
    ├── Weighted / Partial Swap Union-Find
    │   └── Minimize Hamming Distance After Swap Operations [M]
    │       → For each DSU component, count how many source[i] values can be
    │         matched to target[i] values in the group. Unmatched positions
    │         contribute 1 each to Hamming distance
    ├── Connectivity With Constraints
    │   ├── Minimum Cost to Make at Least One Valid Path in a... [M]
    │   │   → For cell (r,c), the free neighbor is determined by grid[r][c]. All
    │   │     other neighbors cost 1. Track dist array initialized to infinity
    │   ├── Remove Max Number of Edges to Keep Graph Fully Tr... [M]
    │   │   → An edge is removable if its union returns False in both relevant
    │   │     DSUs. Final check: both DSUs must reach 1 component, else return -1
    │   ├── Making a Large Island [M]
    │   │   → Label each cell's DSU root. For each 0-cell, collect unique roots of
    │   │     neighboring 1-cells (avoid double-counting same component), sum their
    │   │     sizes
    │   ├── Number of Good Paths [M]
    │   │   → Group nodes by value. For each value group, union all nodes of that
    │   │     value with their neighbors (which have ≤ current value). Count
    │   │     same-value nodes per component root; add k*(k+1)/2 where k = count
    │   └── Largest Component Size by Common Factor [M]
    │       → Nodes are both numbers (index) and prime factors (up to max value).
    │         Use dict-based DSU. For each nums[i], find primes, union nums[i] with
    │         each prime, then count component size for each original number
    └── Union-Find — More Problems
        ├── Redundant Connection II (LC 685, Directed Graph) [M]
        │   → 1. Scan edges; track parent for each node. If a node already has a
        │     parent, record cand1 = prev_edge, cand2 = current_edge and skip cand2.
        │     2. Run Union-Find on remaining edges. If a cycle forms and cand1
        │     exists, return cand1; else return the cycle edge. If no cycle and cand2
        │     exists, return cand2
        └── Smallest String With Swaps (LC 1202) [M]
            → collections.defaultdict(list) — groups[find(i)].append(i) for all i.
              For each group, sort both the indices and the characters, then assign
              characters back
```